
DATA LINK LAYER

UNIT-II

Data link layer:

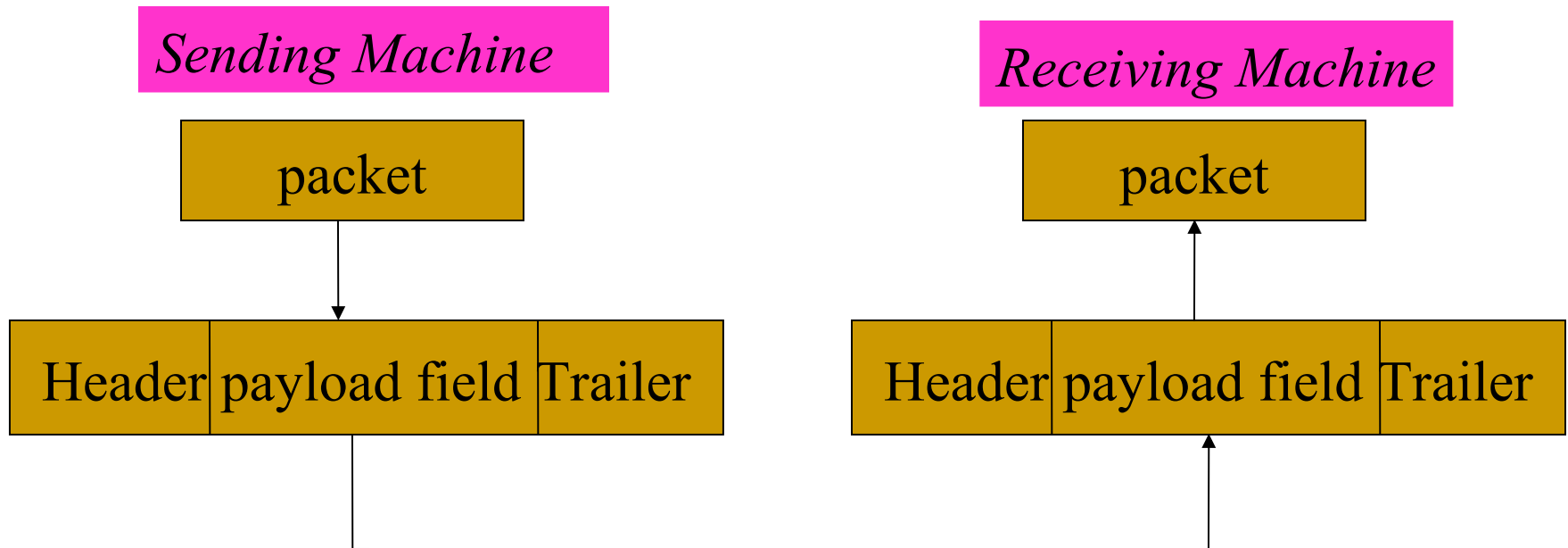
Data-Link Layer has responsibility of transferring frames from one node to adjacent node over a link

■ DATA LINK LAYER DESIGN ISSUES:

- Data link layer has a number of specific functions to carry out. These functions include
 - Provides a *well-defined service interface* to the network layer.
 - Determines how the bits of the physical layer are grouped into frames (*framing*).
 - Deals with transmission errors (*CRC and ARQ*).
 - Systems which automatically request the retransmission of missing packets or packets with errors are called Automatic Repeat reQuest (ARQ)
 - Regulates the flow of frames.

Data link layer-Introduction

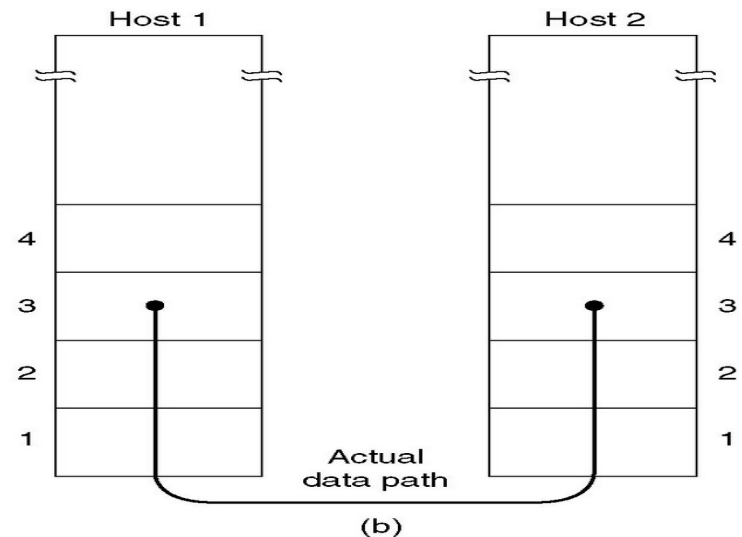
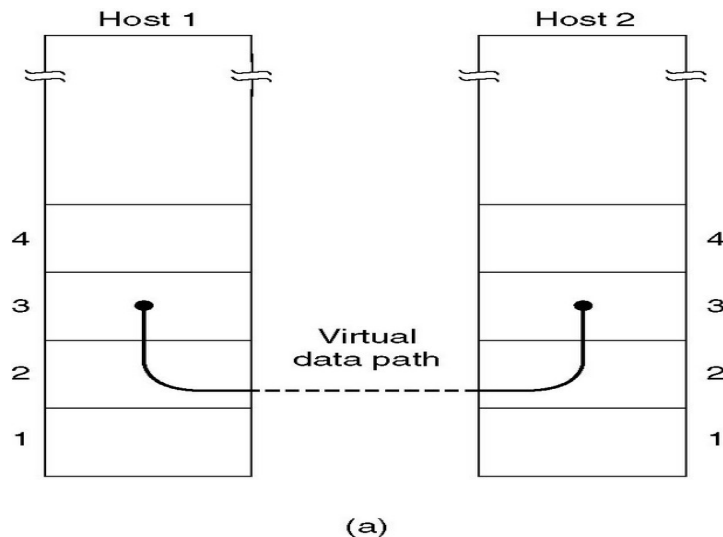
- *The data link layer takes the packets it gets from the network layer and encapsulates them into frames for transmission .*
- *Each frame contains frame header ,a payload field for holding the packet ,and a frame trailer*



Relationship between packets and frames

Data link layer-Introduction

- Services Provided to the Network layer:
 - *The function of the data link layer is to provide services to the network layer*
 - *The principal service is transferring data from the network layer on the source machine to the network layer on the destination machine.*



(a) Virtual Communication

(b) Actual communication

Types of services provided to the Network Layer

- *Unacknowledged Connectionless service*
- *Acknowledged Connectionless service*
- *Acknowledged Connection-Oriented service*

Unacknowledged Connectionless service:

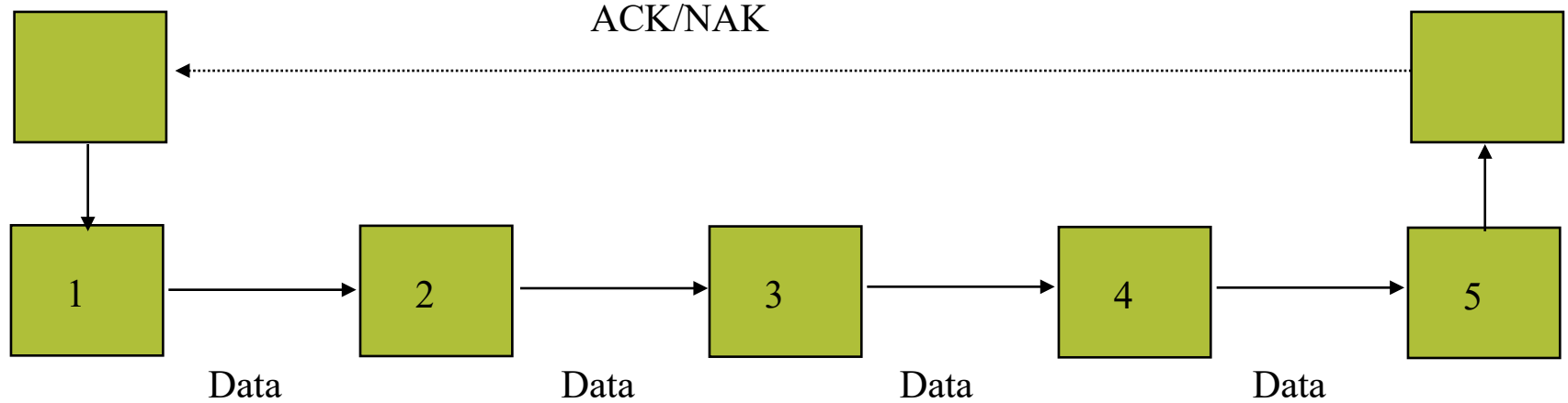
- *It consists of having the source machine send independent frames to the destination machine without having the destination acknowledge them.*
- *No connection is established beforehand or released afterward*
- *If a frame is lost due to noise on the line, no attempt is made to recover it in the data link layer.*
- *Appropriate for voice, where delay is worse than bad data.*
- *Most of the LANs use unacknowledged connection less service in the data link layer*

■ *Acknowledged Connectionless service*

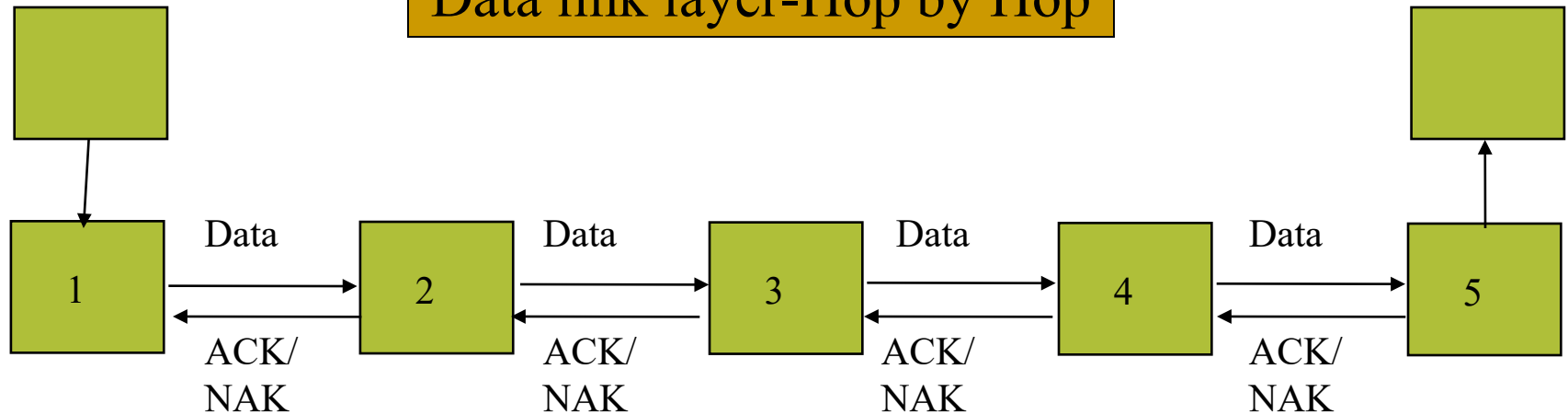
- ❑ *when this service is offered, there are still **no** logical connections used, but each frame sent is individually acknowledged.*
- ❑ *In this way ,the sender knows whether a frame has arrived correctly.*
- ❑ *If it has not arrived within a specified time interval it can be sent again.*
- ❑ *This service is used over unreliable channels ,such as wireless systems.*

- *Acknowledged Connection-Oriented service*
 - ❑ *The most sophisticated service the data link layer can provide to the network layer is connection-oriented service.*
 - ❑ *The source and destination machines establish a connection before any data are transferred*
 - ❑ *Each frame sent over the connection is numbered ,and the data link layer guarantees that each frame is received exactly once and that all frames are received in the right order*
 - ❑ *When connection oriented service is used ,transfers go through three distinct phases.*
 - ❑ *1) Connection established*
 - ❑ *2) Data transferred*
 - ❑ *3) connection Released*

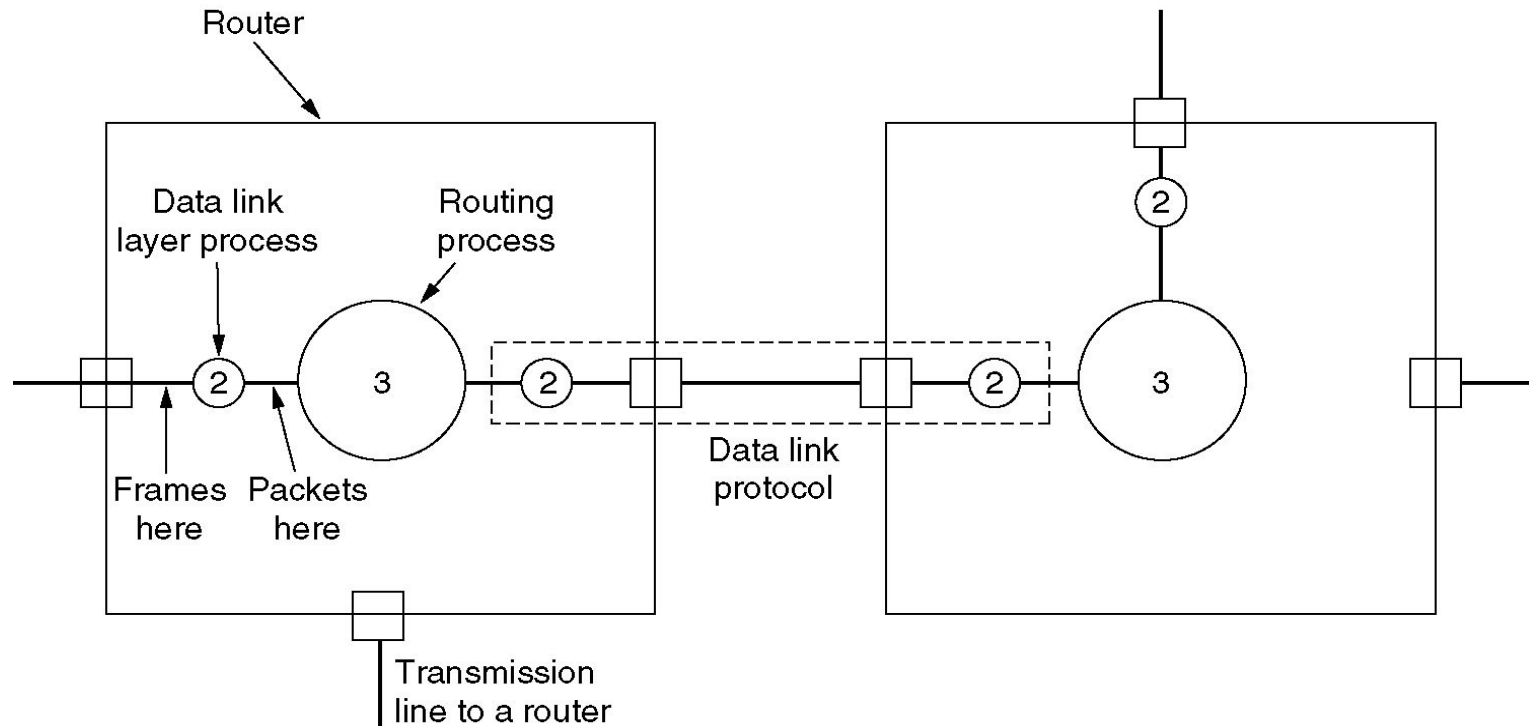
Transport layer-End to End



Data link layer-Hop by Hop



Services Provided to Network Layer



Placement of the Data link protocol

Framing:

- *In order to provide service to the network layer the data link layer must use the **service provided to it by the physical layer**.*
- *Physical layer is used to accept the **raw bit stream** and attempt to deliver it to the destination*
- *The bit stream is not guaranteed to be error free*
- *The number of bits received may be less than ,equal to ,or more than the number of bits transmitted, and they may have different values*
- *It is up to the **data link layer to detect and if necessary ,correct errors**.*
- *The usual approach is for the data link layer to **break the bit stream up into discrete frames and compute the checksum for each frame**.*

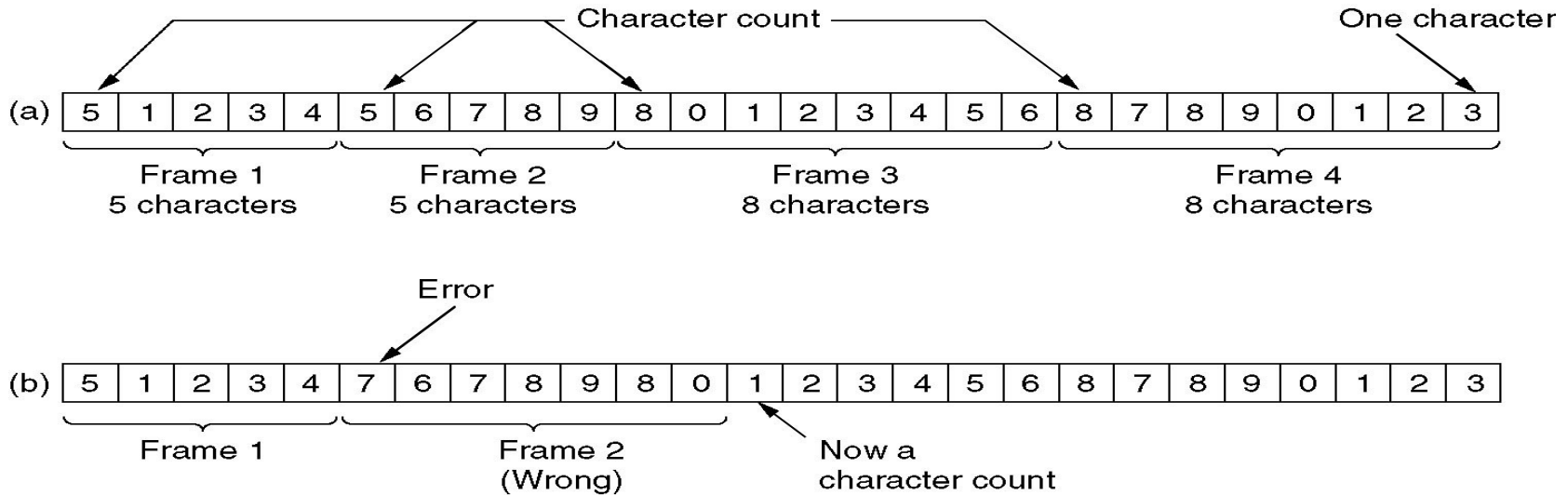
Framing:

- *When a frame arrives at the destination ,the checksum is recomputed.*
- *If the newly computed checksum is different from the one contained in the frame, the data link layer knows that an error has occurred and take steps to deal with it.*
 - *e.g., discarding the bad frame and possibly also sending back an error report*
- *Breaking the bit stream up into frames is more difficult than it at first appears, One way to achieve this framing is to insert time gaps between frames, much like the spaces between words in ordinary text.*

Methods:

- 1) character count*
- 2) Flag bytes with byte stuffing*
- 3) starting and ending flags, with bit stuffing*
- 4) Physical Layer Coding Violations*

Character count:



A character stream a) with out errors b) with one error

- The first framing method uses a field in the header to specify the number of characters in the frame.*
- When the data link layer in the destination sees the character count ,it knows how many characters follow and hence where the end of the frame is.*

Character count:

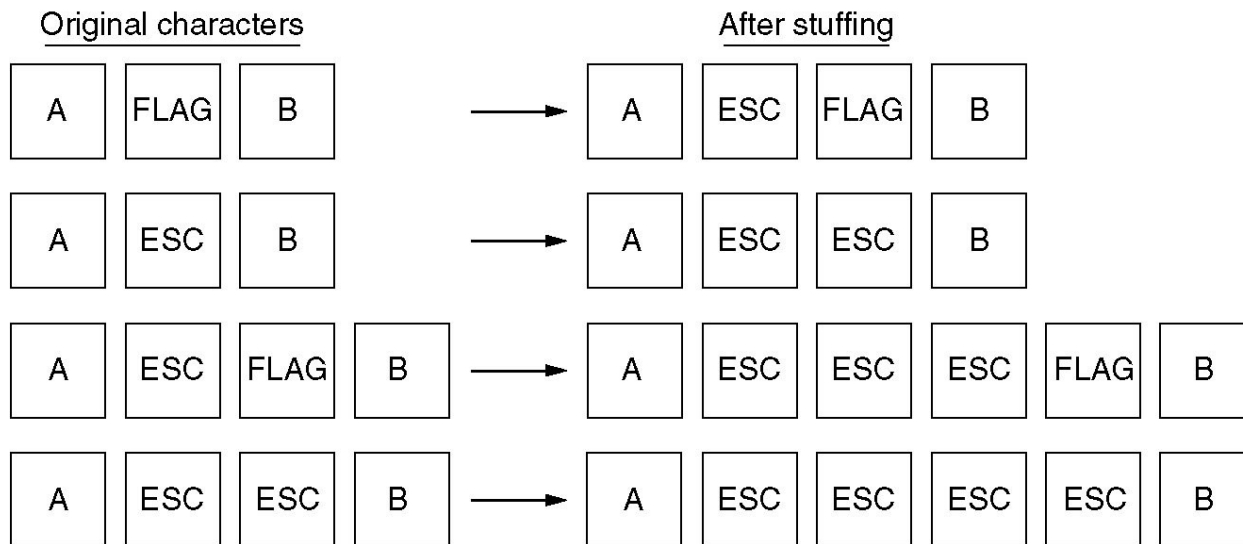
- *The trouble with this algorithm is that the count can be garbled by a transmission error.*
- *For example, if the character count of 5 in the second frame becomes a 7, the destination will get out of synchronization and will be unable to locate the start of the next frame.*
- *Disadvantages:*
 - ❑ *The count can be garbled by the transmission error*
 - ❑ *Resynchronization is not possible. Even if with checksum, the receiver knows that the frame is bad there is no way to tell where the next frame starts.*
 - ❑ *Asking for retransmission doesn't help either because the start of the retransmitted frame is not known*
 - ❑ *No longer used*

- *The second transmission method gets around the problem of **Resynchronization called character stuffing**.*
- *Most protocols used the same byte ,called flag byte, as both the starting and ending delimiter as **FLAG**.*
- *Two consecutive flag bytes indicate the end of one frame and start of the next one.*

- Framing: **Character stuffing** – newer protocols
 - Frame starts / ends with same char: **FLAG**



(a)

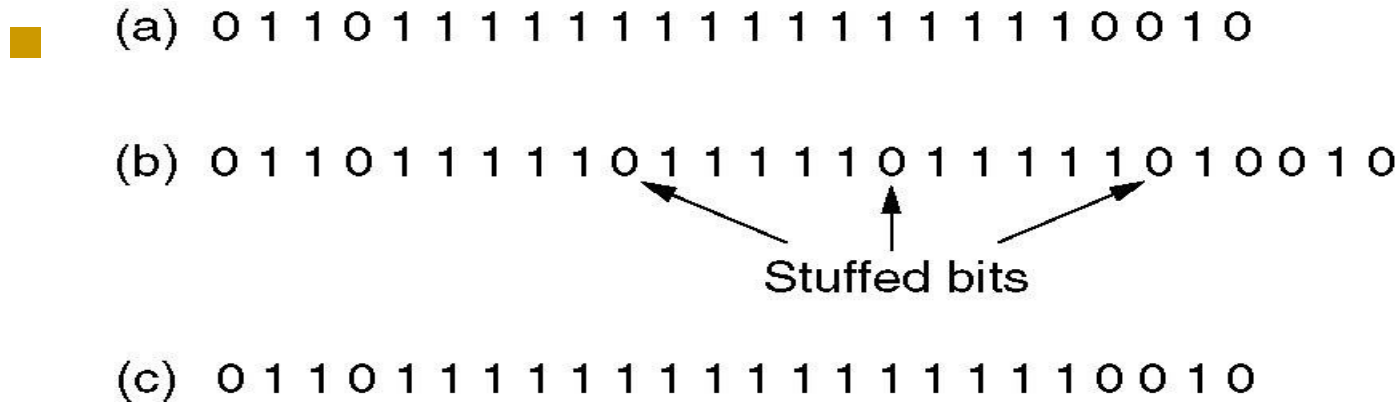


(b)

starting and ending flags, with bit stuffing

- *This new technique allows data frames to contain an arbitrary number of bits and allow character codes with an arbitrary number of bits per character.*
- *Each frame begins and ends with a special bit pattern, 01111110 called a flag byte.*
- *whenever the senders data link layer encounters six consecutive ones in the data , it automatically stuffs a 0 bit into the outgoing bit stream.*
- *whenever the receiver sees five consecutive incoming 1 bits, followed by a 0 bit ,it automatically destuffs (i.e deletes) the 0 bit.*
- *If the flag pattern 01111110, this flag is transmitted as 0111111010 but stored in the receiver memory as 01111110.*

starting and ending flags, with bit stuffing

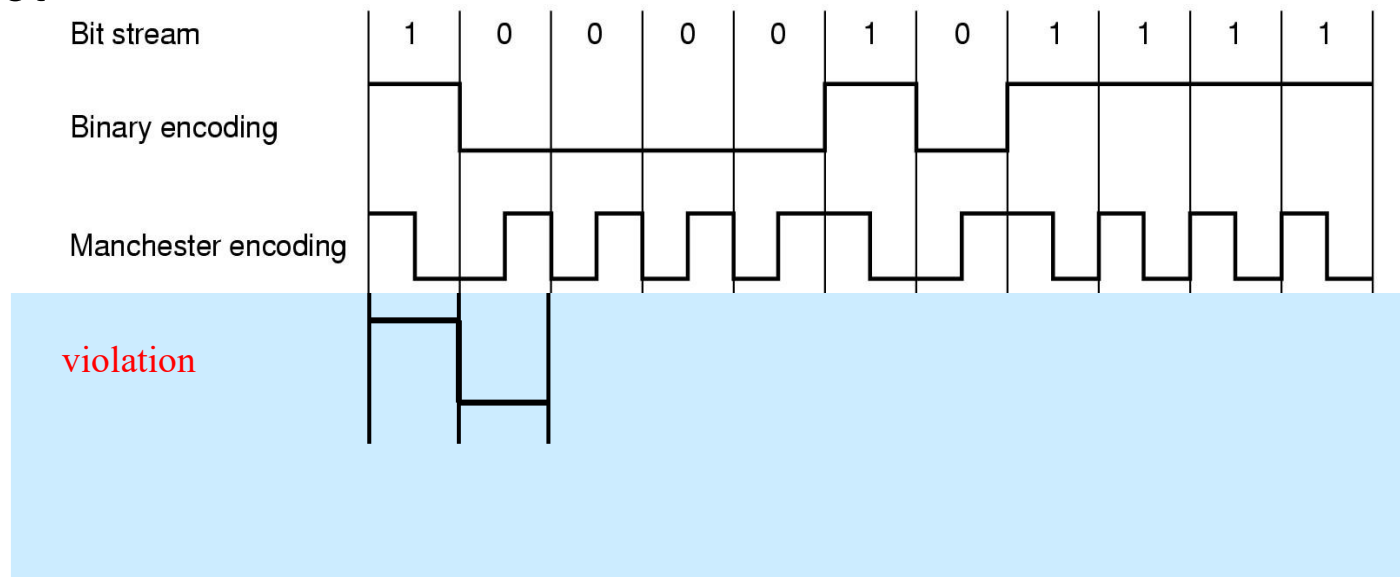


Bit stuffing

- (a) *The original data.*
- (b) *The data as they appear on the line.*
- (c) *The data as they are stored in receiver's memory after destuffing.*

- Framing: **Coding violation**

- Redundancy in the encoding on medium is required
- e.g. Manchester encoding: transition in the middle of a slot



- Use no transition in a slot (= **coding violation**) as start of frame

Error Control:

- *Next problem: How to make sure all frames are eventually delivered to the network layer at the destination, and in the proper order.*
 - *The usual way to ensure **reliable delivery** is to provide the sender with some feedback about what is happening at the other end of the line.*
 - *The protocol calls for the receiver to send back special **control frames** bearing **positive and negative acknowledgements** about the incoming frames*
-

Error Control:

- *If the sender receives a **positive acknowledgement** about a frame ,it knows the frame has arrived safely.*
 - *on the other hand, a **negative acknowledgement** means that something has gone wrong, and the frame must be transmitted again.*
 - *An additional complication comes from hardware troubles.*
 - *Managing the **Timers and sequence numbers** so as to ensure that each frame is ultimately passed to the network layer at the destination **exactly once**.*
-

Error Control:

- *Maintaining Timers for Error Control: When a sender transmits a frame, it generally also starts a timer .*
- *The timer is set to expire after an interval long enough for the frame to reach the destination, be processed there ,and have the acknowledgement to propagate back to the sender.*
- *Normally, the frame will be correctly received and the acknowledgement will get back before the timer runs out ,in which case the timer will be cancelled.*
- *However, if either the frame or the acknowledgement is lost ,the timer will go off ,alerting the sender to a potential problem .the obvious solution is to just transmit the frame again.*
- *Sequence number is used to recognize the duplicate packet.*

Flow control:

- *A sender that systematically wants to **transmit frames faster than the receiver** can accept them..*
- *When the sender is running on a **fast (or lightly loaded)** computer and the receiver is running on a **slow (or heavily loaded machine)***
- *The sender keeps pumping the frames out at a higher rate until the receiver is completely swamped.*
- *Even if the transmission is error free ,at a certain point the receiver will simply be unable to handle the frames as they arrive and will start to lose someone*
- *To prevent this situation **Feedback- based flow control & Rate-based flow control mechanisms are used.***
- ***Feedback- based flow control** ,the receiver sends back information to the sender giving it permission to send more data .*

-
- Rate-based flow control focuses on regulating the data transmission rate, typically by setting a predefined limit on how quickly the sender can transmit data. This limit is often determined based on the receiver's capacity to process the data.
-

Error Detection and correction:

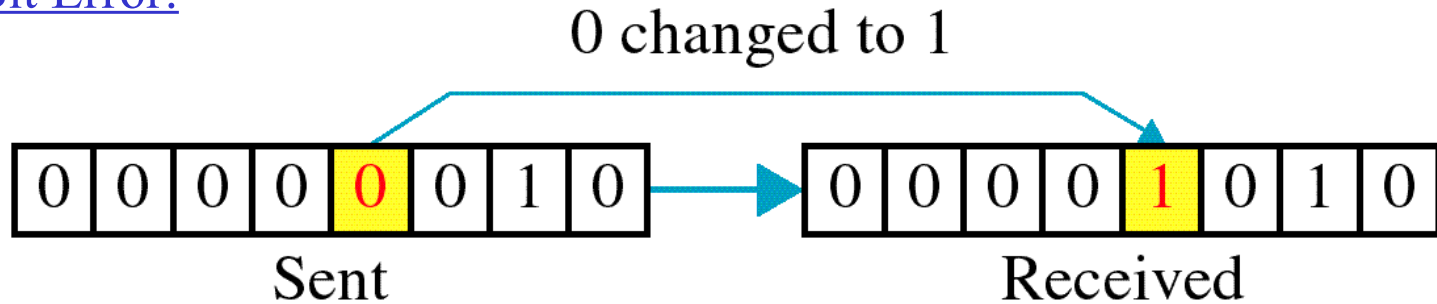
- *In some cases it is sufficient to detect an error and in some, it requires the errors to be corrected also.*
- *For e.g.*
 - *On a reliable medium : Error Detection is sufficient where the error rate is low and asking for retransmission after Error Detection would work efficiently*
 - *In contrast, on an unreliable medium : Retransmission after Error Detection may result in another error and still another and so on. Hence Error Correction is desirable.*

Error Detection and Correction

- Types of Errors
 - Error-Correcting Codes
 - Error-Detecting Codes
-

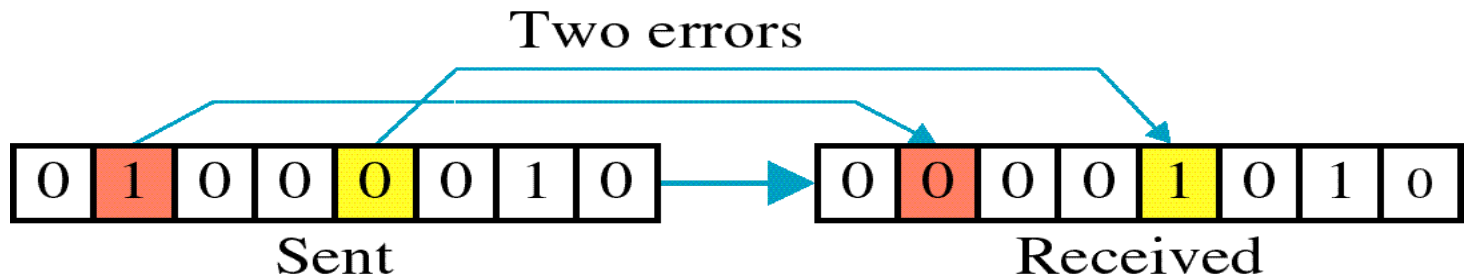
Types of Errors

Single-Bit Error:

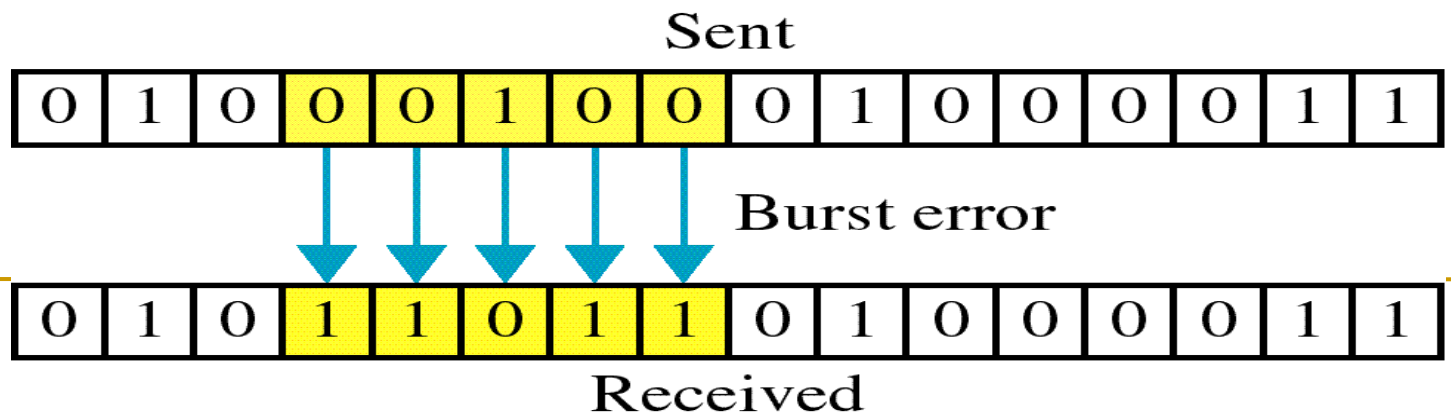


Burst Error:

(a)

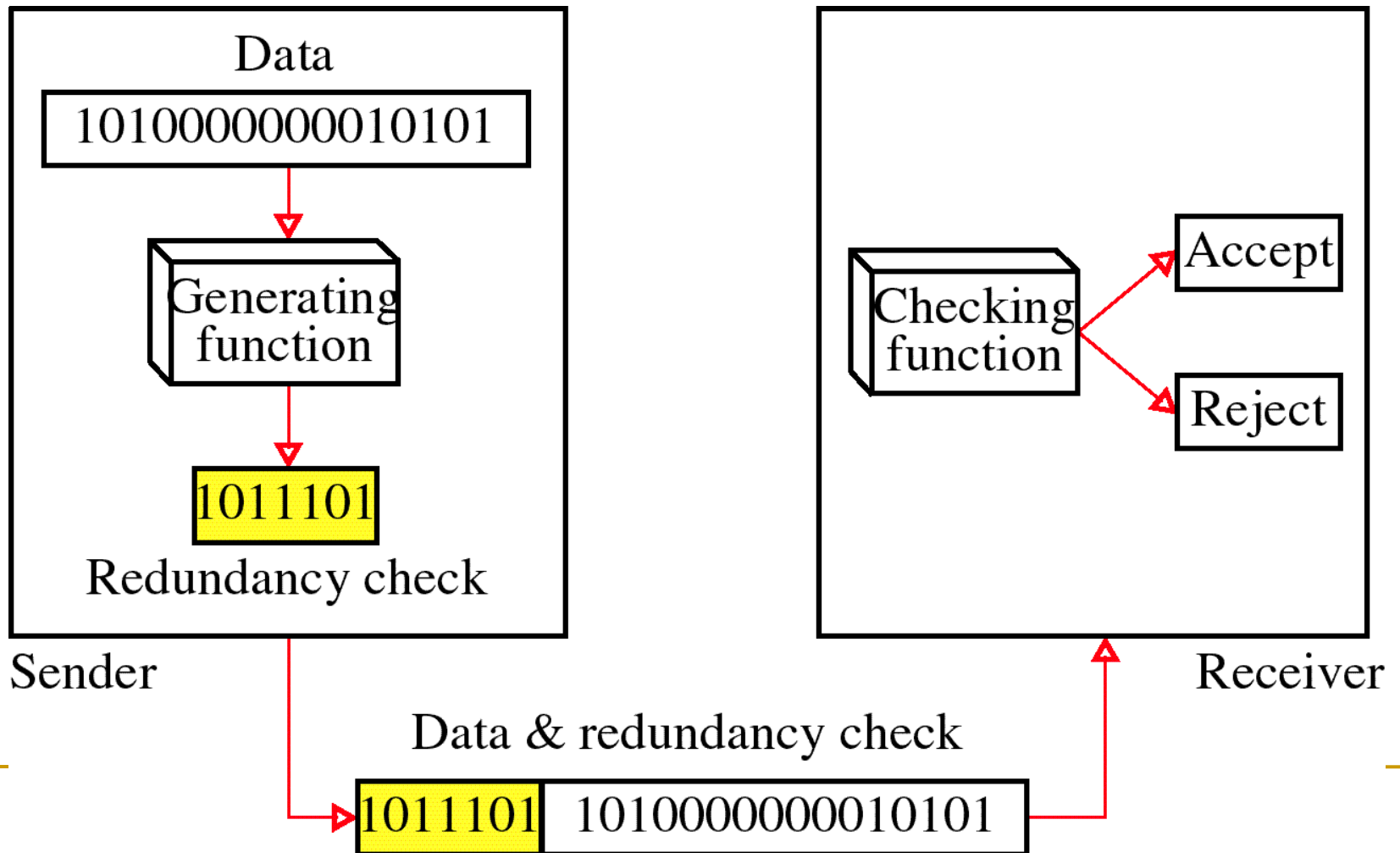


(b)



Redundancy

→ All error detection/correction methods are based on redundancy



Error detecting Codes:

An error detection code is a binary code that detects digital errors during transmission.

There are 3 ways in which we can detect errors in the received message :

- 1. Parity Bit**
- 2. Check Sum**
- 3. Cyclic Redundancy Check (CRC)**

-
- **What is parity bit?**
 - A parity bit is an extra bit that is added to the message bits or data-word bits on the sender side. Data-word bits along with parity bits is called a codeword. The parity bit is added to the message bits on the sender side, to help in error detection at the receiver side.
-

Even Parity

- Total number of 1's in the given data bit should be even. So if the total number of 1's in the data bit is odd then a single 1 will be appended to make total number of 1's even else 0 will be appended.

Odd Parity

- In odd parity system, if the total number of 1's in the given binary string (or data bits) are even then 1 is appended to make the total count of 1's as odd else 0 is appended.

Comparison:

- The receiver compares the calculated parity with the expected parity(based on whether even or odd parity was used in transmission)
 - Error Indication:
 - If the received parity does not match the expected parity
-

Data Frame

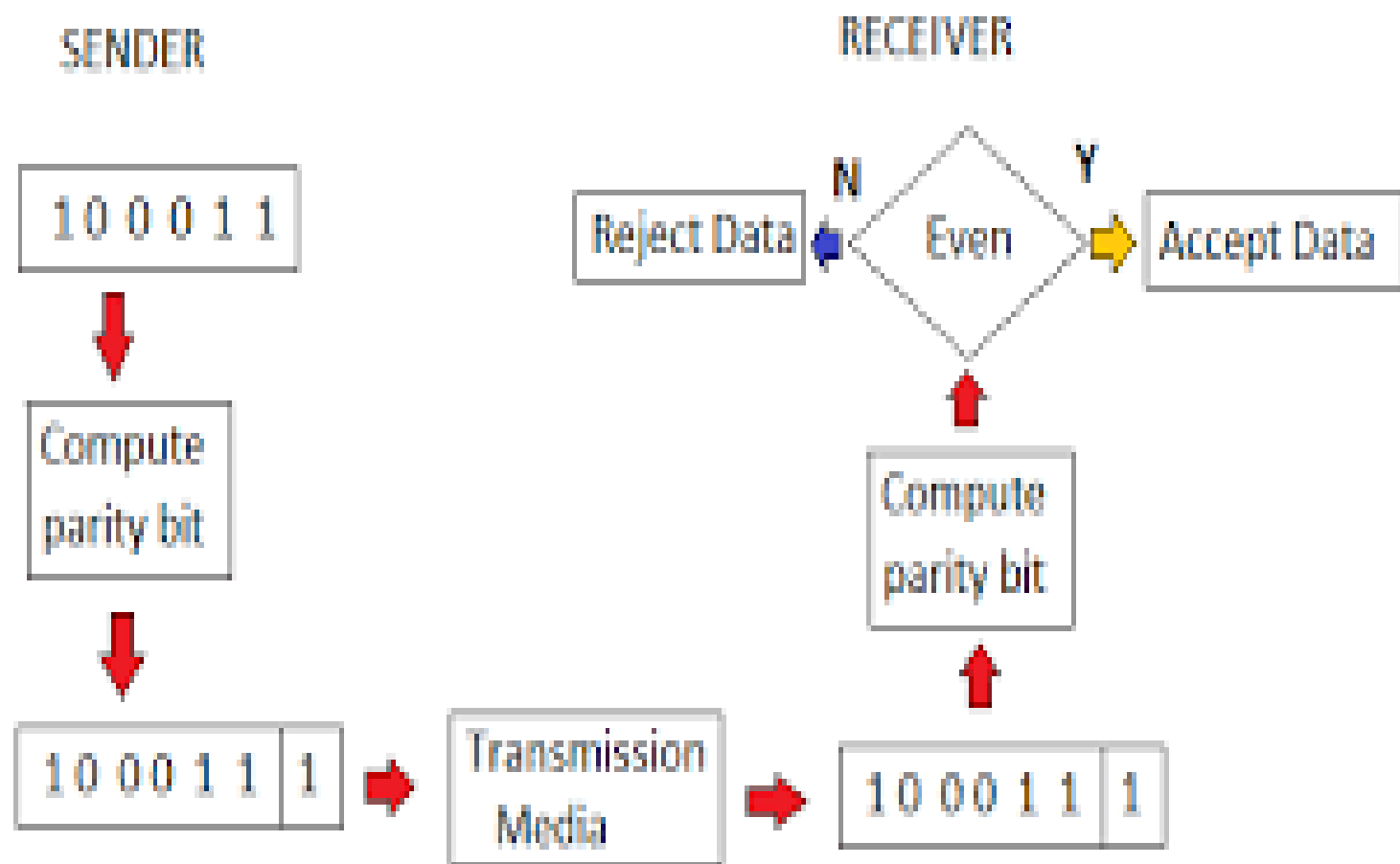


Frame with
even parity bit



Frame with
odd parity bit





Cyclic Redundancy Check

- Cyclic Redundancy Check or CRC is a method of detecting errors in the communication channel.
- CRC uses **Generator Polynomial** which is available on both sender and receiver side.
- An example generator polynomial is of the form like $x^3 + x + 1$. This generator polynomial represents key 1011. Another example is $x^2 + 1$ that represents key 101.

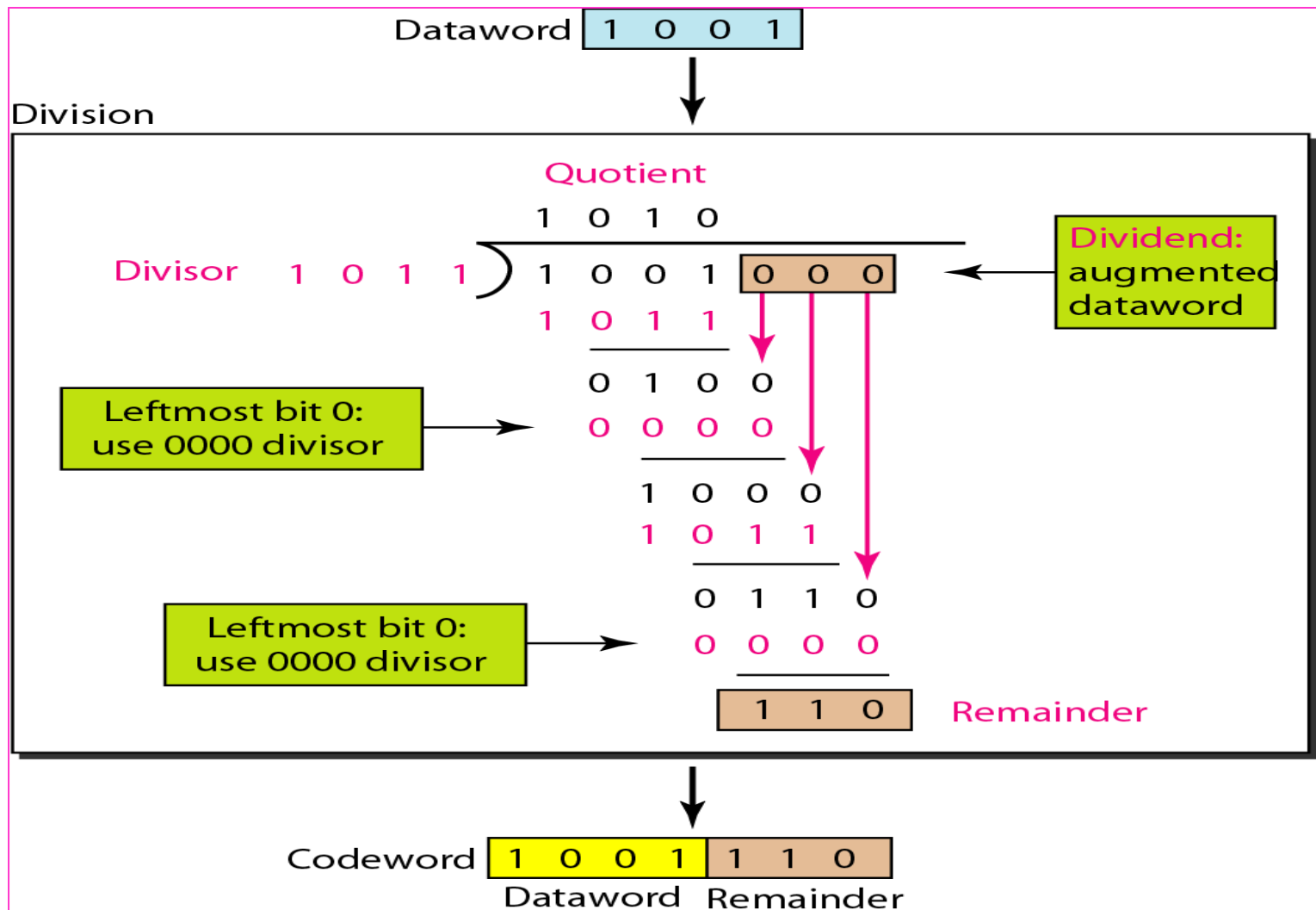
-
- There are two primary variables in CRC:
 - **n**: Number of bits in data to be sent from sender side
 - **k**: Number of bits in the key obtained from generator polynomial.
-

-
- **Encoded Data Generation from Generator Polynomial (Sender Side)**
 - The binary data is first augmented by adding $k-1$ zeros in the end of the data.
 - Then, modulo - 2 binary division is used to divide binary data by the key and remainder of division is stored.
 - At last the the remainder is appended at the end of the data to form the encoded data which is later sent.
-

Checking Error in Transmission (Receiver Side)

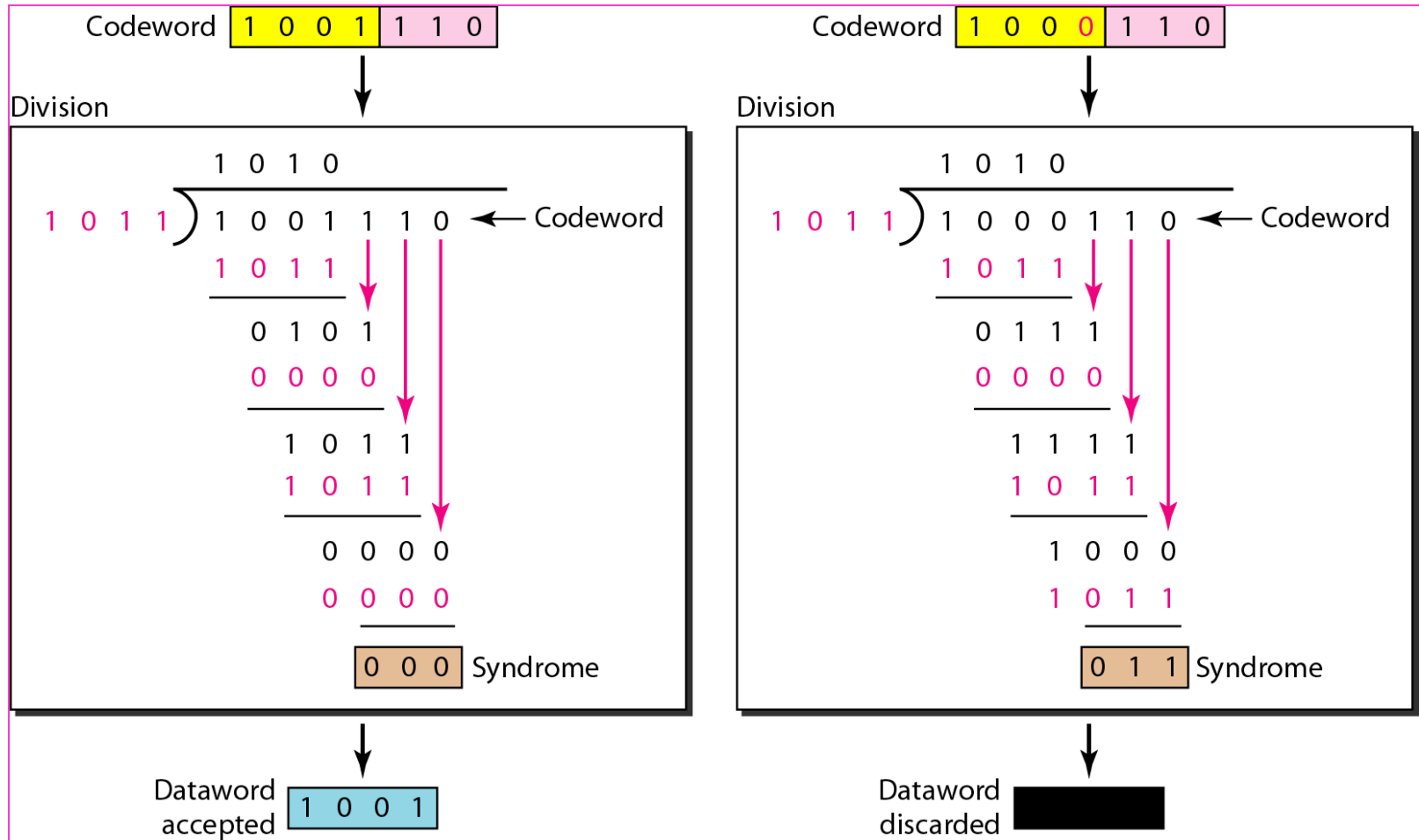
- After receiving the data, to check if the data is error free, perform the modulo-2 division again.
 - If the remainder is 0, then there are not errors, otherwise, the data is faulty and contain transmission errors.
-

Calculation of the polynomial code checksum:



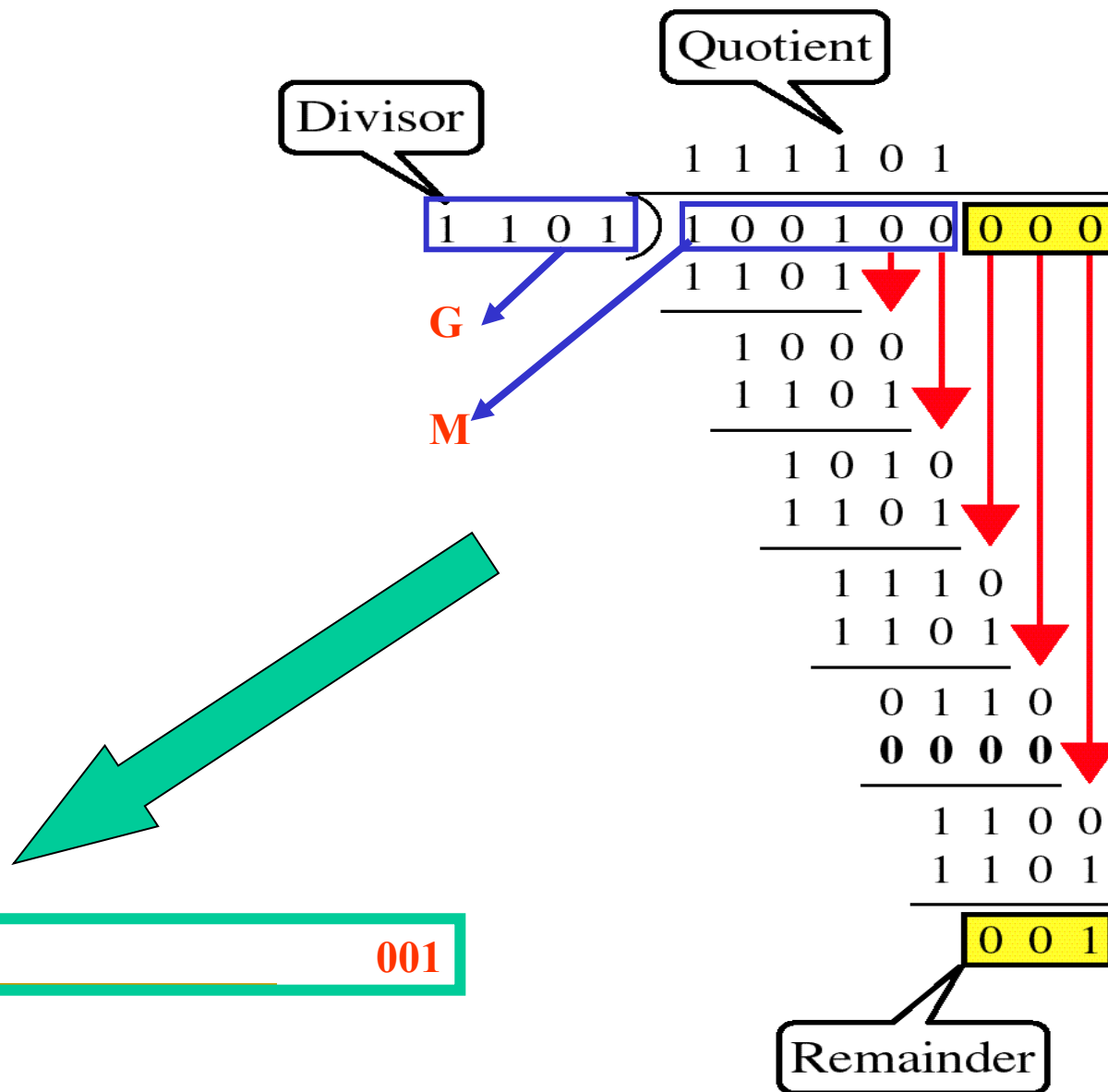
Frame: 1001 Generator : 1011

Division in the CRC decoder for two cases:



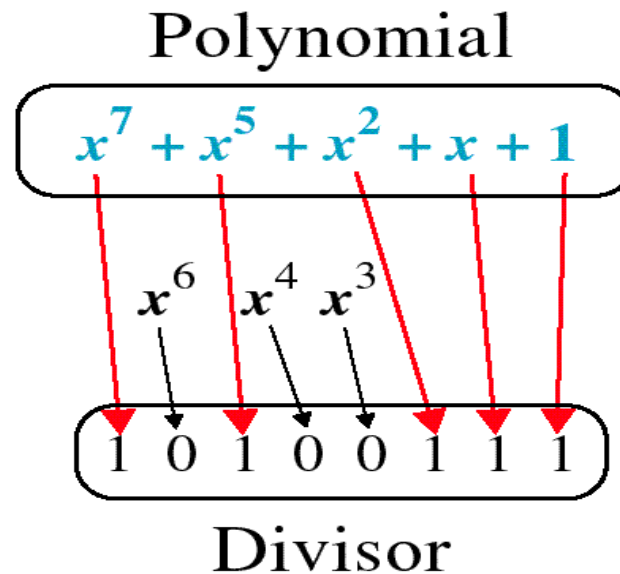
CRC

Example:



CRC

Why polynomials?



Standard polynomials:

CRC-12

$$x^{12} + x^{11} + x^3 + x + 1$$

CRC-16

$$x^{16} + x^{15} + x^2 + 1$$

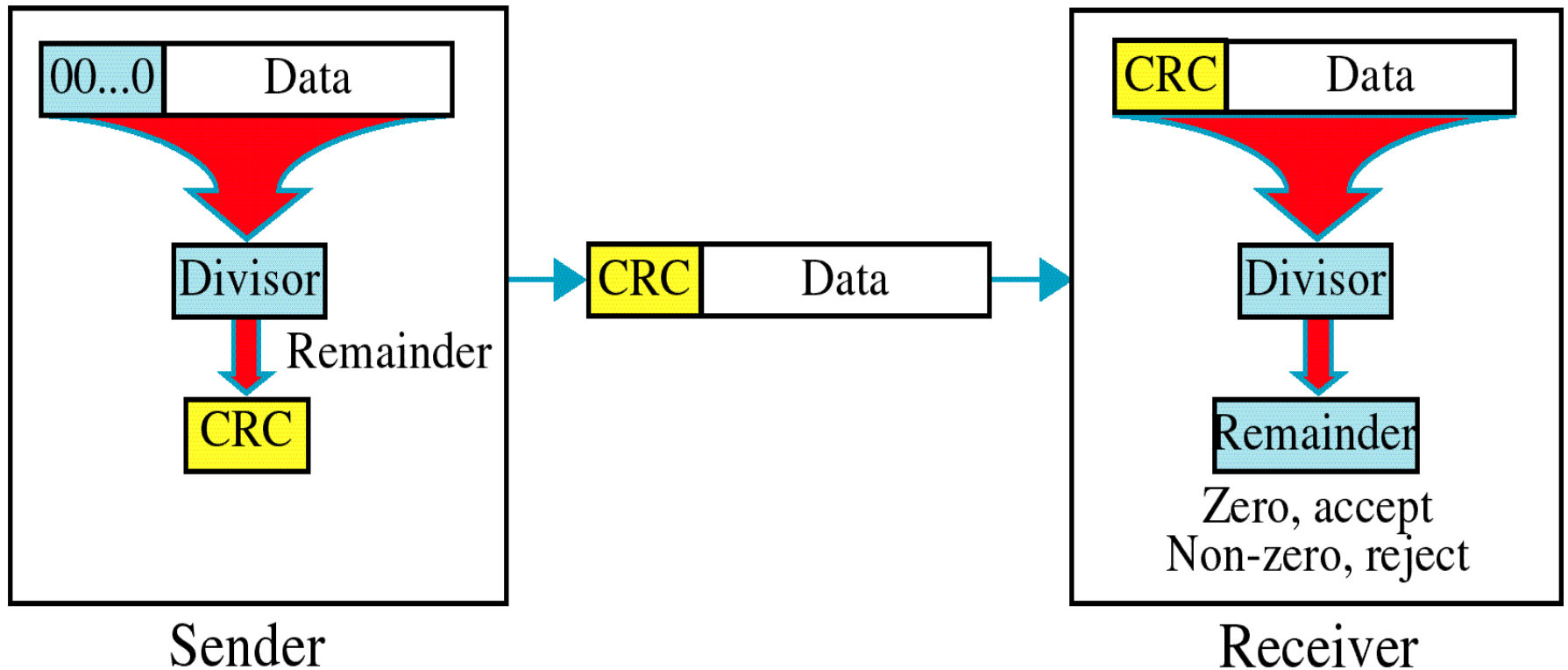
CRC-ITU

$$x^{16} + x^{12} + x^5 + 1$$

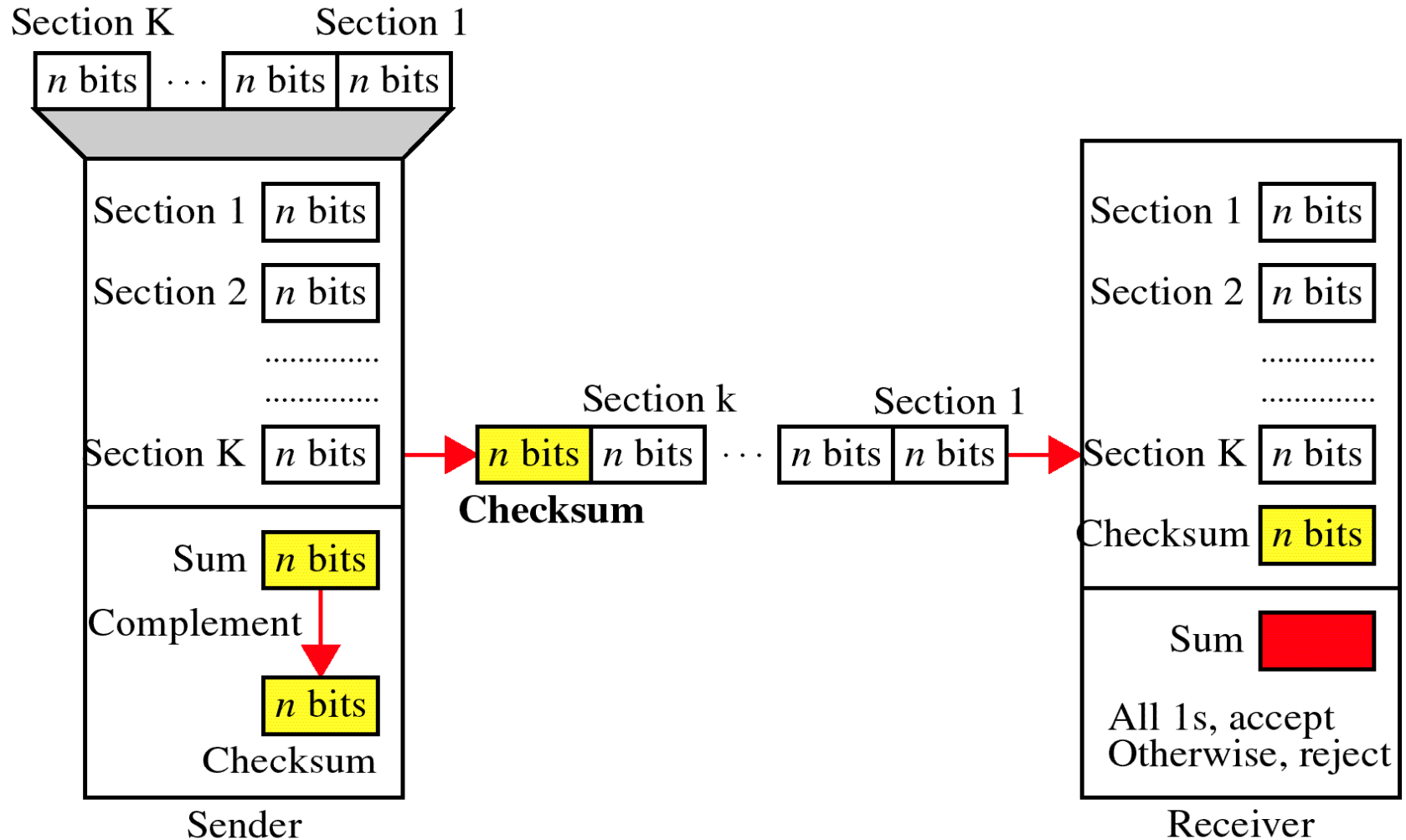
CRC-32

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

CRC



Checksum



At the sender

- ➡ The unit is divided into k sections, each of n bits.
- ➡ All sections are added together using one's complement to get the sum.
- ➡ The sum is complemented and becomes the checksum.
- ➡ The checksum is sent with the data

Performance

- ➔ The checksum detects all errors involving an odd number of bits.
 - ➔ It detects most errors involving an even number of bits.
 - ➔ If one or more bits of a segment are damaged and the corresponding bit or bits of opposite value in a second segment are also damaged, the sums of those columns will not change and the receiver will not detect a problem.
-

At the receiver

- ➡ The unit is divided into k sections, each of n bits.
- ➡ All sections are added together using one's complement to get the sum.
- ➡ The sum is complemented.
- ➡ If the result is zero, the data are accepted: otherwise, they are rejected.

Case	A	+	B	Sum	Carry
1	0	+	0	0	0
2	0	+	1	1	0
3	1	+	0	1	0
4	1	+	1	0	1

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

10101001

00111001

Sum 11100010

Checksum **00011101**

The pattern sent is 10101001 00111001 **00011101**

Original Data

10011001	11100010	00100100	10000100
----------	----------	----------	----------

1

2

3

4

k=4, m=8

Sender

```

1  10011001
2  11100010
   -----
   ①01111011
   ↘      1
     01111100
3  00100100
   -----
     10100000
4  10000100
   -----
   ①00100100
   ↘      1
     00100101
Sum: 00100101
Checksum: 11011010
  
```

Reciever

```

1  10011001
2  11100010
   -----
   ①01111011
   ↘      1
     01111100
3  00100100
   -----
     10100000
4  10000100
   -----
   ①00100100
   ↘      1
     00100101
     11011010
Sum: 11111111
Complement: 00000000
Conclusion: Accept Data
  
```

Hamming Distance

- The Hamming distance between two words of same size is the no.of differences between corresponding bits.
- The Hamming distance can be found if we apply XOR operation on the two words and count no.of 1's in the result.
- $d(000,011)$ is 2 because $(000) \text{ XOR } (011) = 011$ (two 1s)
- When a code word is corrupted during transmission, the hamming distance between sent and received code words is the number of bits affected by the error.
- For e.g. if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is $d(00000,01101)=3$.

Error correcting codes:

- Minimum distance for Error Detection...
- Let us find minimum distance in a code if we want to be able to detect upto s errors.
- If s errors occur during transmission, the Hamming distance between the sent codeword and received codeword is s .
- If our code is to detect up to s errors, the minimum distance between valid code words be $s + 1$, so that the received codeword does not match a valid codeword.
- In other words, if the minimum distance between all valid code words is $s + 1$, the received codeword cannot be erroneously mistaken for another codeword.

Error correcting codes:

- The distances are not enough($s+1$) for the receiver to accept it as valid. The error will be detected.
- A code with $d_{\min}=s+1$ may be able to detect more than s errors in some special cases, only s or fewer errors are guaranteed to be detected.
- To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{\min}=s+1$.
- A code scheme has $d_{\min}=4$. What is error detection and correction capability of this scheme?
- This scheme guarantees detection of upto three errors, but can correct up to one error

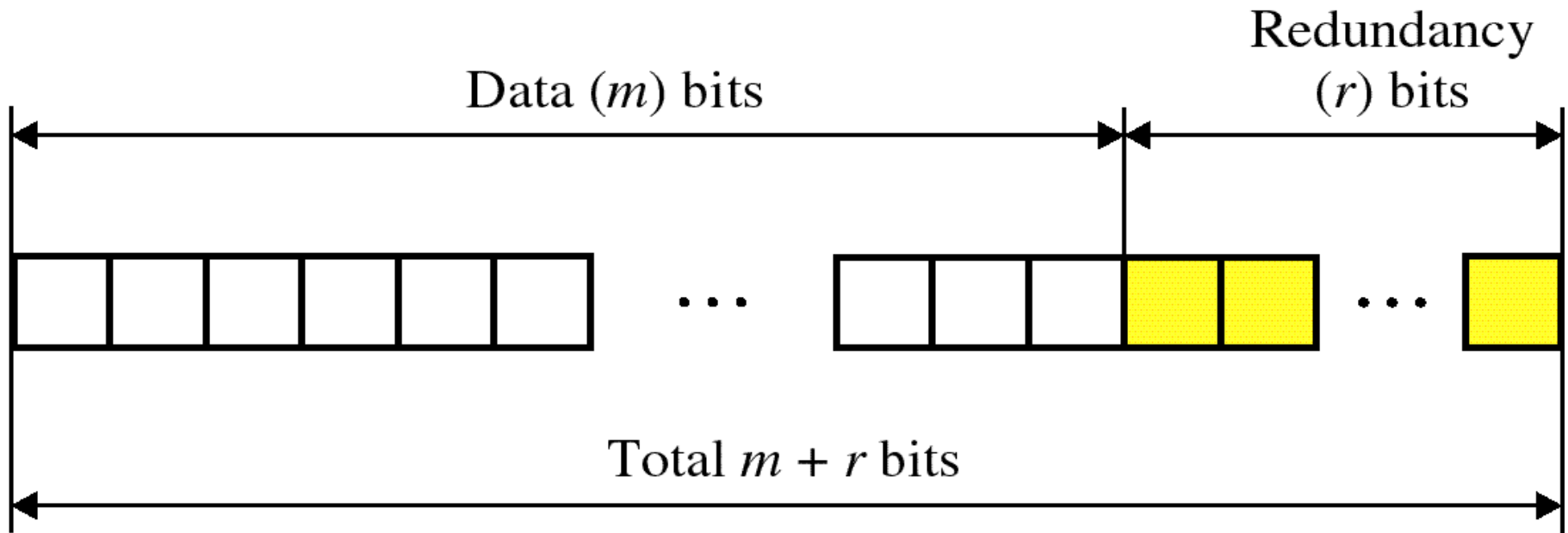
Error correcting codes:

- **Forward Error Correction:** FEC is the only error correction scheme that actually detects and corrects transmission errors at the receive end without calling for retransmission.
 - Ex: **Hamming code**
- No of bits in hamming code is dependent on the no. of bits in the data character by using the relation.
 - $2^n \geq m + n + 1$
 - Where n = no. of hamming bits
 - m = no. of bits in the data character
- Ex: For a 12-bit data stream of 101100010010, determine the hamming bits, assume a single bit in error & prove how hamming code detects it.

Error correcting codes:

- No. of bits in hamming code
- $2^n \geq m+n+1$
 - Take $n=4$
 - $2^4=16 \geq 12+4+1$
 - $16 \geq 17$
- Take $n=5$
 - $2^5=32 \geq 12+5+1=18$
 - $32 \geq 18$ ---- 5 bits are required for hamming code.
 - 12 bits of data +5 bits of code=17 bits of data stream.

Error correcting codes:



1. Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).
2. All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).
3. All the other bit positions are marked as data bits.
4. Each data bit is included in a unique set of parity bits, as determined its bit position in binary form.
 - a. Parity bit 1 covers all the bits positions whose binary representation includes a 1 in the least significant position (1, 3, 5, 7, 9, 11, etc).
 - b. Parity bit 2 covers all the bits positions whose binary representation includes a 1 in the second position from the least significant bit (2, 3, 6, 7, 10, 11, etc).
 - c. Parity bit 4 covers all the bits positions whose binary representation includes a 1 in the third position from the least significant bit (4–7, 12–15, 20–23, etc).

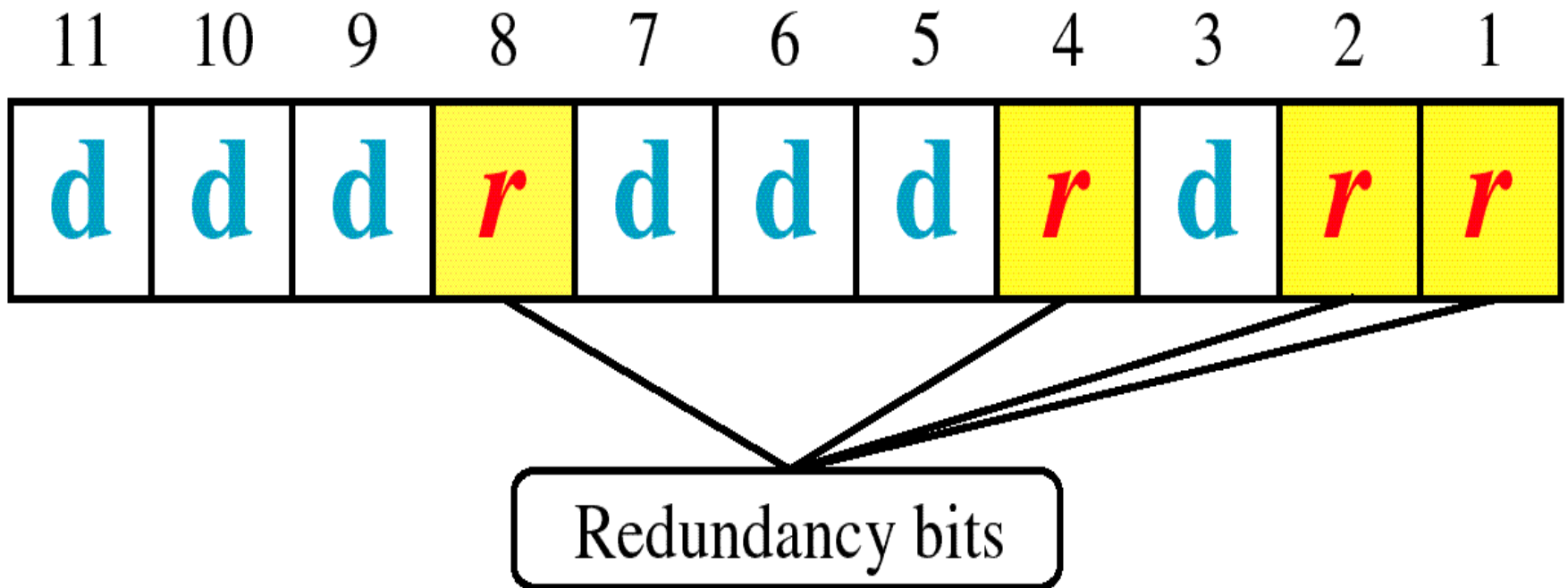
d. Parity bit 8 covers all the bits positions whose binary representation includes a 1 in the fourth position from the least significant bit bits (8–15, 24–31, 40–47, etc).

e. In general each parity bit covers all bits where the bitwise AND of the parity position and the bit position is non-zero.

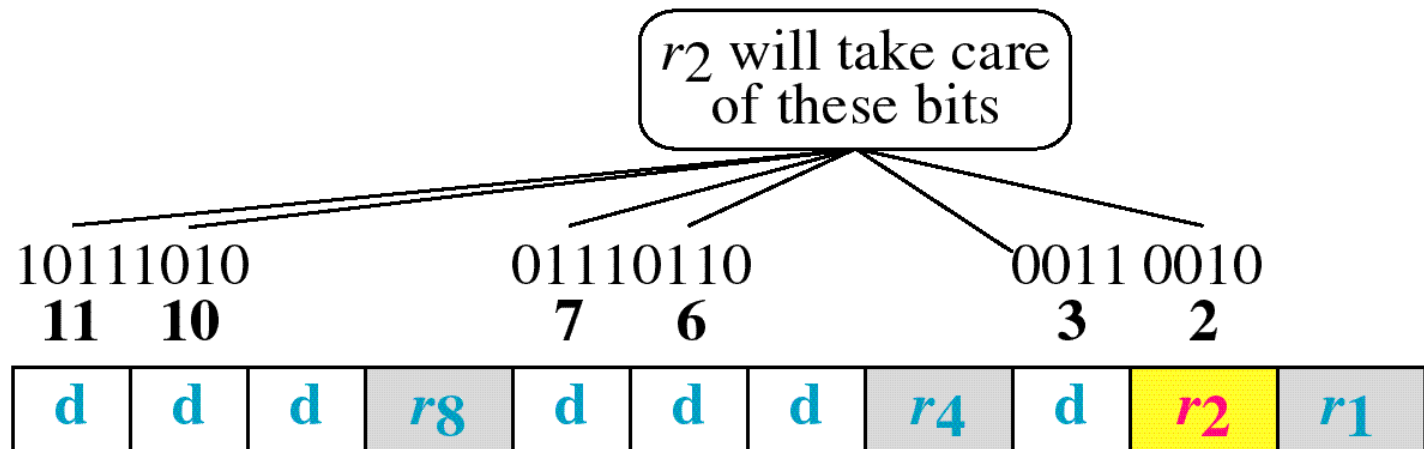
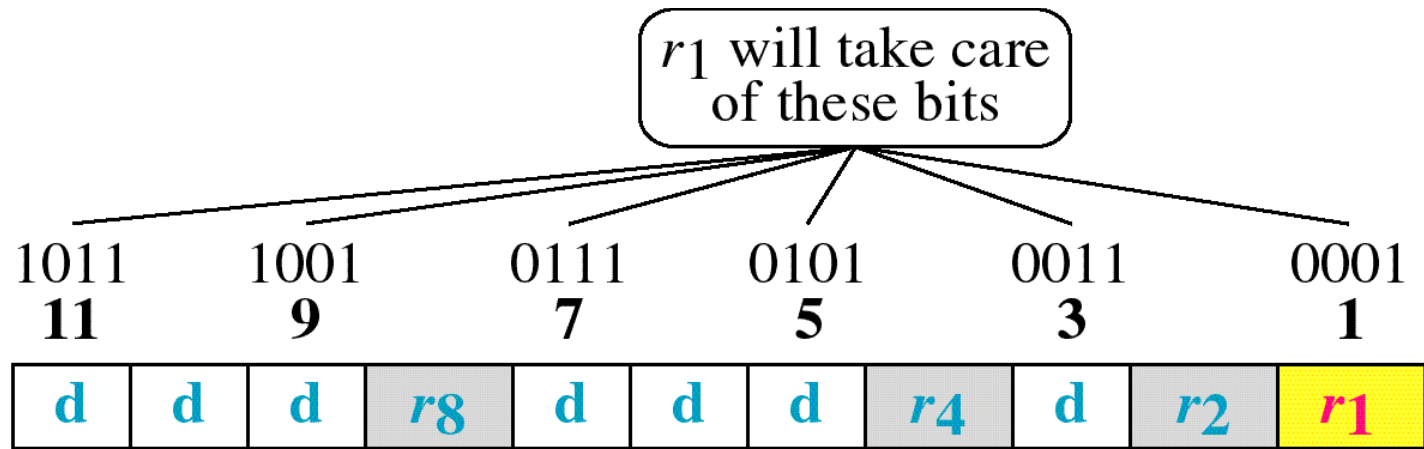
Since we check for even parity set a parity bit to 1 if the total number of ones in the positions it checks is odd.

Set a parity bit to 0 if the total number of ones in the positions it checks is even

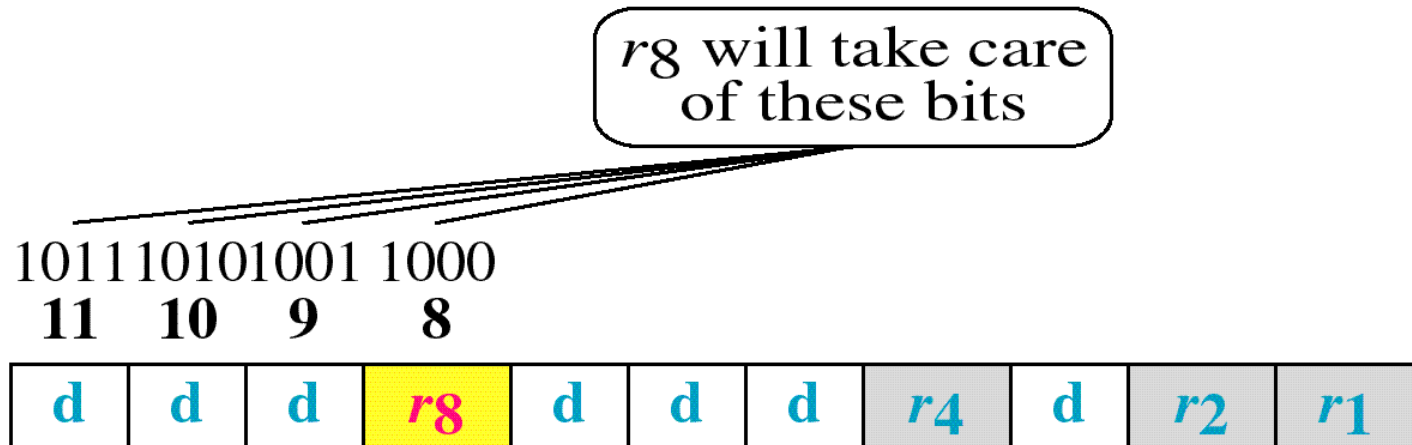
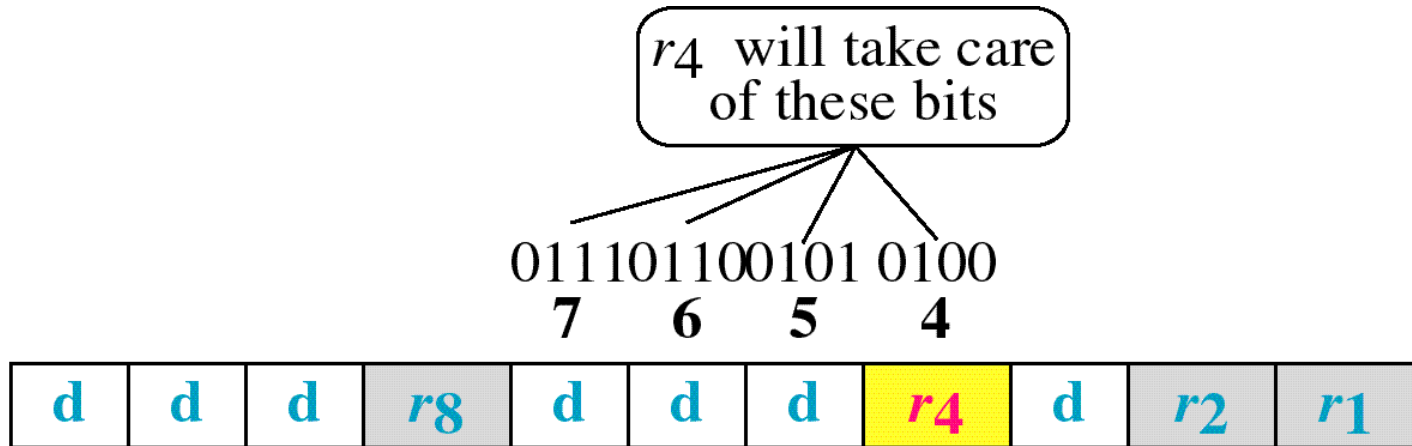
Hamming Code



Hamming Code



Hamming Code



Example of Hamming Code

Data: 1 0 0 1 1 0 1



Data

1	0	0		1	1	0		1		
---	---	---	--	---	---	---	--	---	--	--

Adding r_1

1	0	0		1	1	0		1		1
---	---	---	--	---	---	---	--	---	--	---

Adding r_2

1	0	0		1	1	0		1	0	1
---	---	---	--	---	---	---	--	---	---	---

Adding r_4

1	0	0		1	1	0	0	1	0	1
---	---	---	--	---	---	---	---	---	---	---

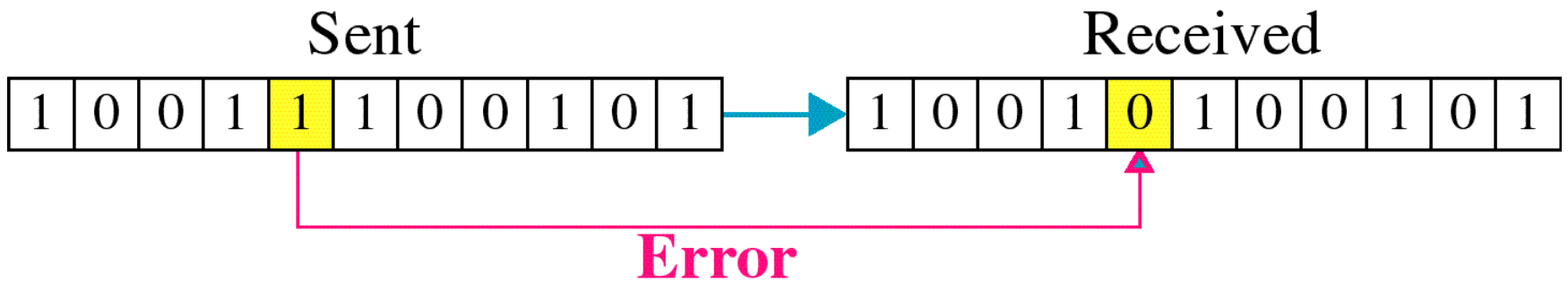
Adding r_8

1	0	0	1	1	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---

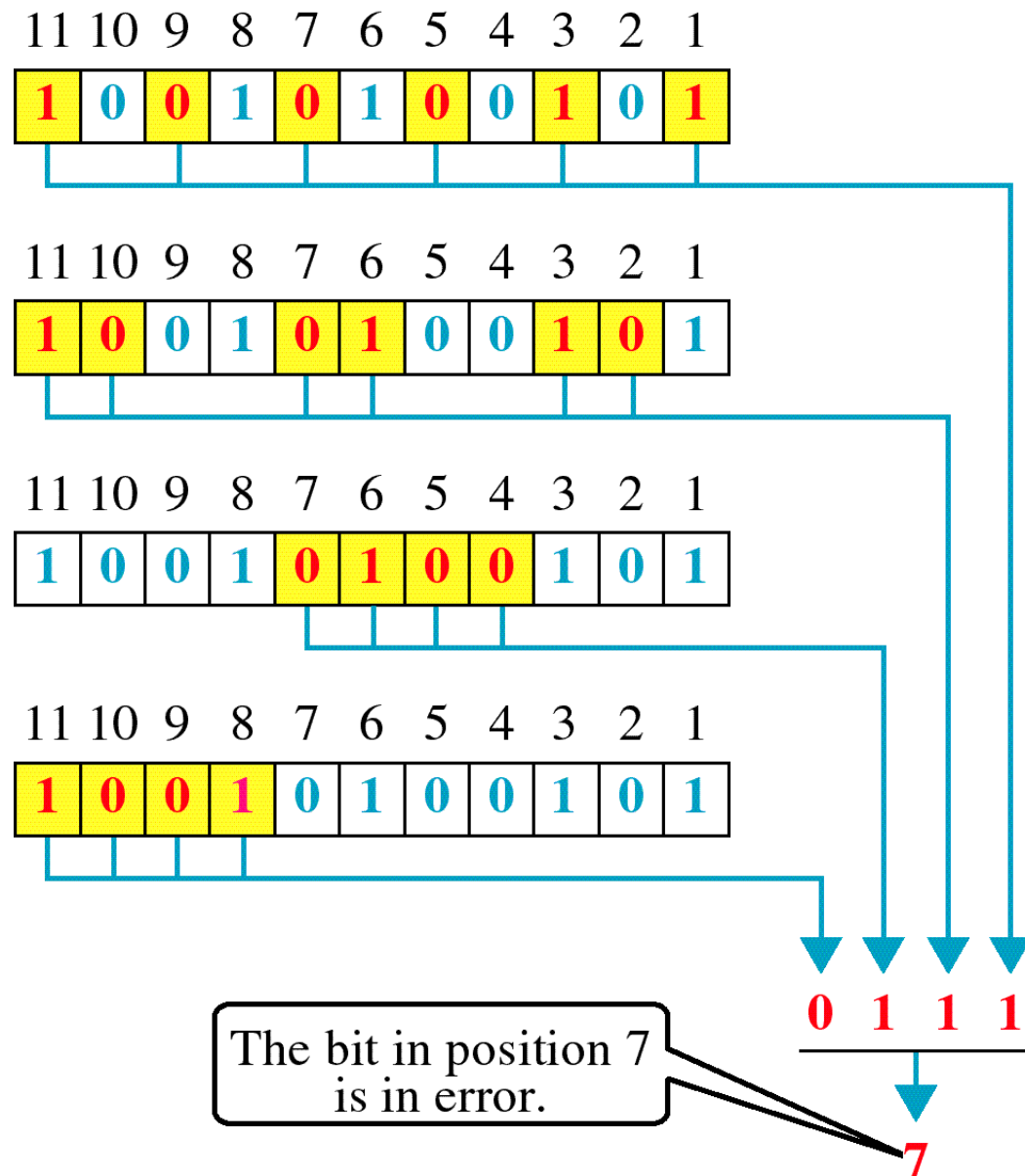


Code: 1 0 0 1 1 1 0 0 1 0 1

Single-bit error



Error Detection



Hamming Code: Alternate method

- *consider the example $m = 1101110$*
 - $2^n \geq m + n + 1$
 - $n = 4$ i.e. $2^4 \geq 7 + 4 + 1$
 - $16 \geq 12$
 - *4 hamming bits needed.*

11	10	9	8	7	6	5	4	3	2	1
1	1	0	r	1	1	1	r	0	r	r

11	10	9	8	7	6	5	4	3	2	1
1	1	0	r	1	1	1	r	0	r	r

Express all bit positions that contains a 1 as four bit number and xor them together.

Bit position

binary number

5

0101

6

0110

Xor

0011

7

0111

xor

0100

10

1010

xor

1110

11

1011

0101

Error correcting codes:

- *Hamming code: 0101*
- *positions: b1=1 b2=0 b4=1 b8=0*
- *The 11 bit data stream is*



11	10	9	8	7	6	5	4	3	2	1
1	1	0	r	1	1	1	r	0	r	r
1	1	0	0	1	1	1	1	0	0	1

Error correcting codes:

- *proof: If the receiver receive an error at 7th position i.e 1 is misinterpreted as 0, then the error message received in.*



11	10	9	8	7	6	5	4	3	2	1
1	1	0	0	1	1	1	1	0	0	1
				↓						
				0						

The garbled data is: 11000111001

Error correcting codes:

- At receiver: hamming code 0101

5

0101

xor

0000

6

0110

xor

0110

10

1010

xor

1100

11

1011

0111

Error correcting codes:

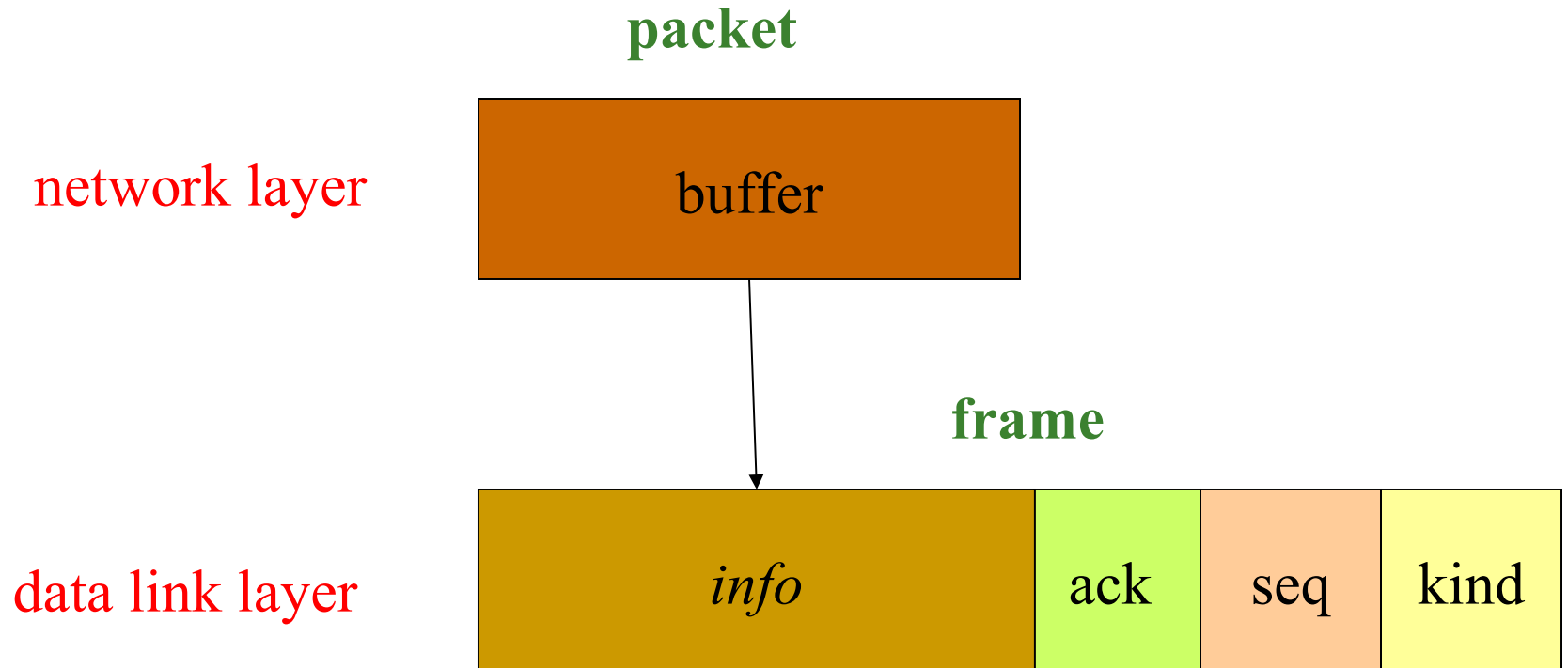


Char.	ASCII	Check bits
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101

Order of bit transmission

Use of a Hamming code to correct burst errors.

Frame header of Data link layer



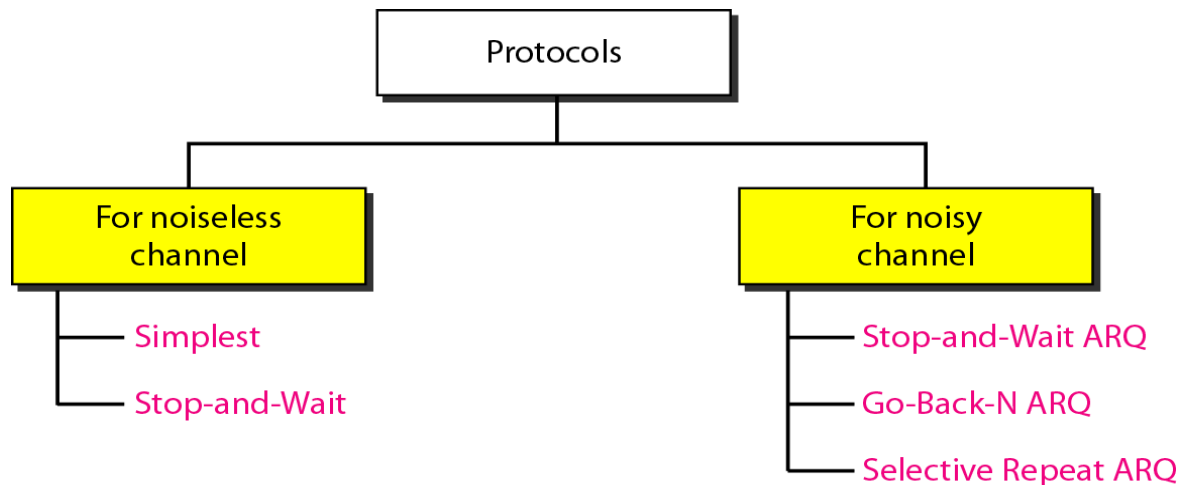
Flow and Error Control

Data link control = flow control + error control

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgement

Error control in the data link layer is based on *automatic repeat request* (ARQ), which is the retransmission of data

ACK, NAK(Negative ACK), Piggybacking (ACKs and NAKs in data frames)



Elementary Data link Protocols:

■ Protocols:

- ❑ 1) *unrestricted **simplex** protocol*
- ❑ 2) *A simplex **stop-and-wait** protocol*
- ❑ 3) *A simplex protocol for a **Noisy Channel***

Protocol Definitions

```
#define MAX_PKT 1024                                /* determines packet size in bytes */

typedef enum {false, true} boolean;                  /* boolean type */
typedef unsigned int seq_nr;                          /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet; /* packet definition */
typedef enum {data, ack, nak} frame_kind;             /* frame_kind definition */

typedef struct {                                     /* frames are transported in this layer */
    frame_kind kind;                                /* what kind of a frame is it? */
    seq_nr seq;                                     /* sequence number */
    seq_nr ack;                                     /* acknowledgement number */
    packet info;                                    /* the network layer packet */
} frame;
```

Continued →

Some definitions needed in the protocols to follow.
These are located in the file protocol.h.

Protocol Definitions (ctd.)

Some definitions
needed in the
protocols to follow.
These are located in
the file protocol.h.

```
/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack_timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack_timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network_layer_ready event. */
void enable_network_layer(void);

/* Forbid the network layer from causing a network_layer_ready event. */
void disable_network_layer(void);

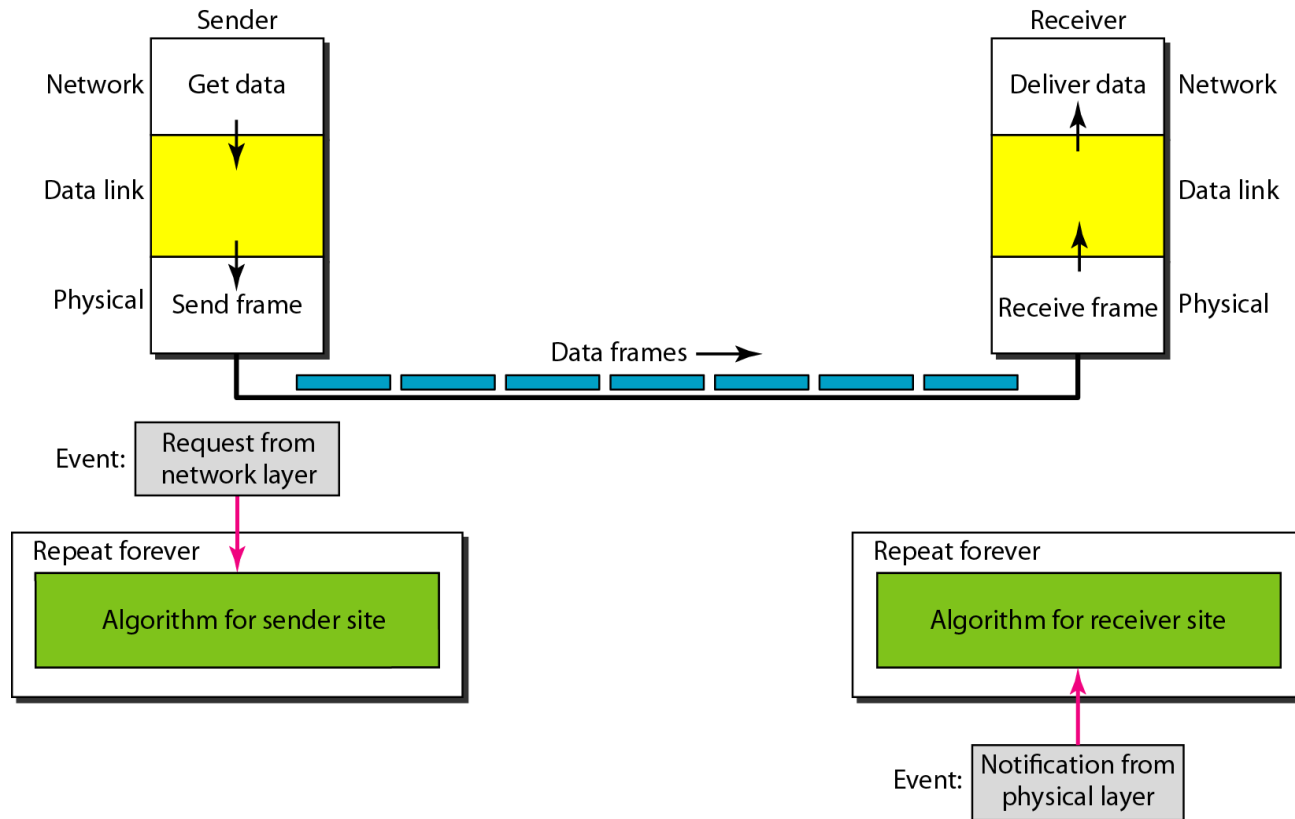
/* Macro inc is expanded in-line: Increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

unrestricted simplex protocol

- *Data are transmitted in one direction only.*
- *Both the transmitting and receiving network layers are always ready*
- *processing time can be ignored*
- *Infinite buffer space is available*
- *The protocol consists of two distinct procedures, a sender and a receiver. The sender runs in the data link layer of the source machine and the receiver runs in the data link layer of the destination machine.*
- *MAX-SEQ is not used because No sequence numbers and Acknowledgements are used here.*
- *The only event type possible is frame-arrival (i.e. an arrival of undamaged frame)*

Noiseless Channels: Simplest Protocol

Simplest protocol with no flow or error control



```
/* Protocol 1 (utopia) provides for data transmission in one direction only, from
sender to receiver. The communication channel is assumed to be error free,
and the receiver is assumed to be able to process all the input infinitely quickly.
Consequently, the sender just sits in a loop pumping data out onto the line as
fast as it can. */
```

```
typedef enum {frame arrival} event type;
#include "protocol.h"
```

```
void sender1(void)
```

```
{
    frame s;                                /* buffer for an outbound frame */
    packet buffer;                          /* buffer for an outbound packet */

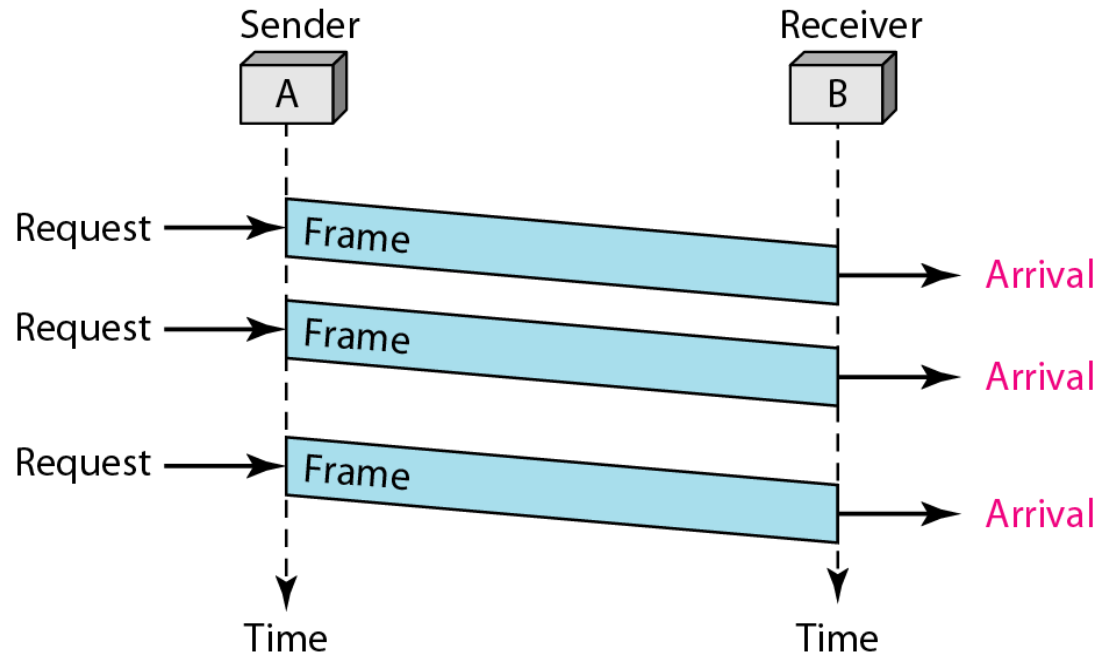
    while (true) {
        from_network_layer(&buffer); /* go get something to send */
        s.info = buffer;             /* copy it into s for transmission */
        to_physical_layer(&s);       /* send it on its way */
    }
    /* Tomorrow, and tomorrow, and tomorrow,
       Creeps in this petty pace from day to day
       To the last syllable of recorded time
       - Macbeth, V, v */
}
```

```
void receiver1(void)
```

```
{
    frame r;
    event_type event; /* filled in by wait, but not used here */

    while (true) {
        wait_for_event(&event); /* only possibility is frame_arrival */
        from_physical_layer(&r); /* go get the inbound frame */
        to_network_layer(&r.info); /* pass the data to the network layer */
    }
}
```

Flow diagram for Noiseless Channels: Simplest Protocol

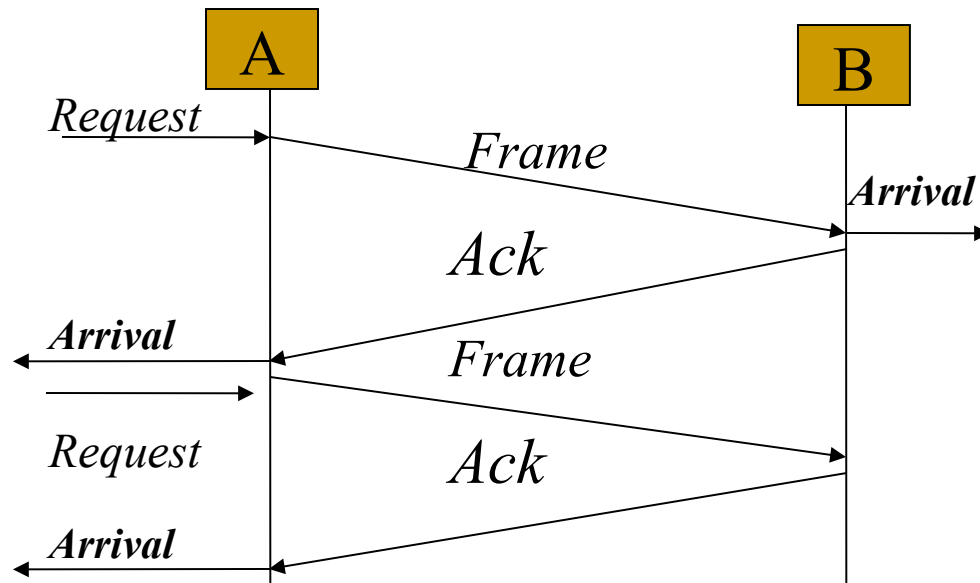


A simplex stop-and-wait protocol:

- *If data frames arrives at the receiver side faster than they can be processed, the frames must be stored until their use*
- *Normally ,the receiver does not have enough storage space ,especially if it is receiving data from many sources*
- *This may result in the discarding frames*
- *To prevent this there must be feedback from the receiver to the sender.*

A simplex stop-and-wait protocol:

- *Def : Protocol in which the sender sends one frame and then waits for acknowledgement before proceeding is called stop-and-wait.*
- *Flow Diagram:*



/* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from sender to receiver. The communication channel is once again assumed to be error free, as in protocol 1. However, this time, the receiver has only a finite buffer capacity and a finite processing speed, so the protocol must explicitly prevent the sender from flooding the receiver with data faster than it can be handled. */

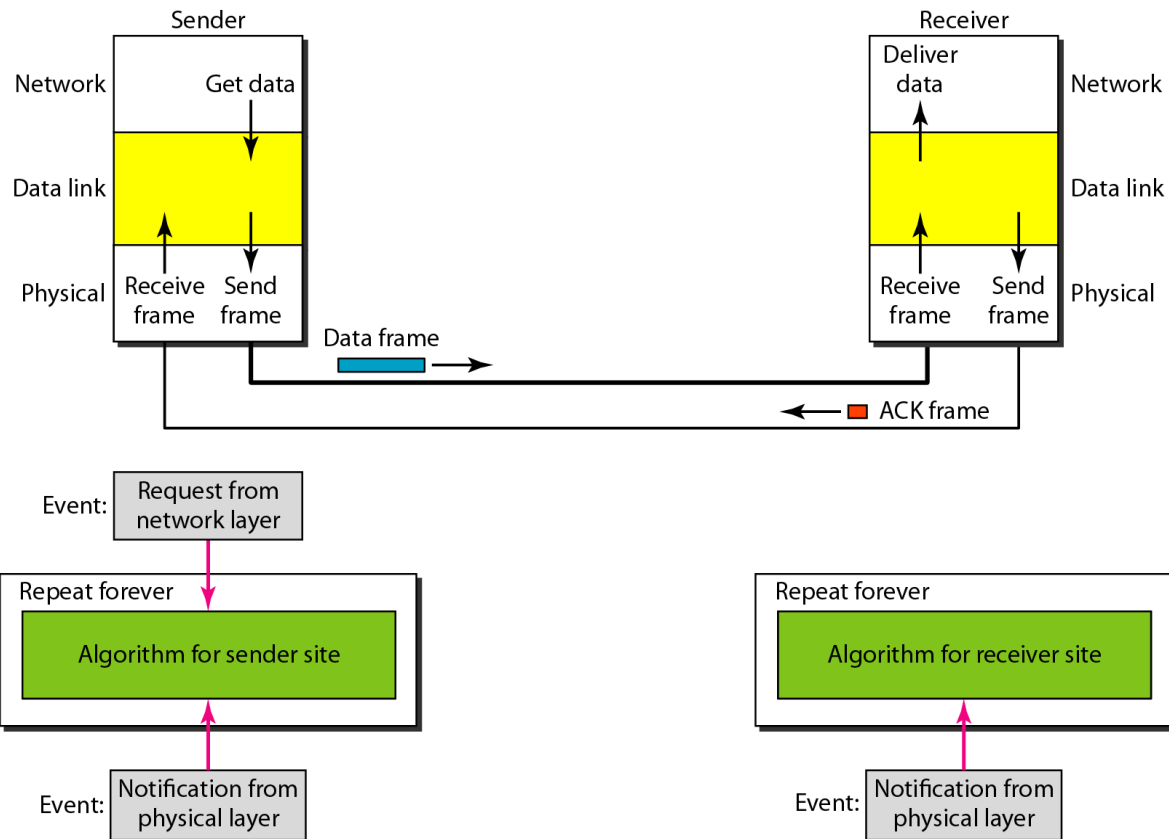
```
typedef enum {frame_arrival} event_type;  
#include "protocol.h"
```

```
void sender2(void)  
{  
    frame s;                /* buffer for an outbound frame */  
    packet buffer;          /* buffer for an outbound packet */  
    event_type event;       /* frame_arrival is the only possibility */  
  
    while (true) {  
        from_network_layer(&buffer); /* go get something to send */  
        s.info = buffer;             /* copy it into s for transmission */  
        to_physical_layer(&s);       /* bye bye little frame */  
        wait_for_event(&event);      /* do not proceed until given the go ahead */  
    }  
}
```

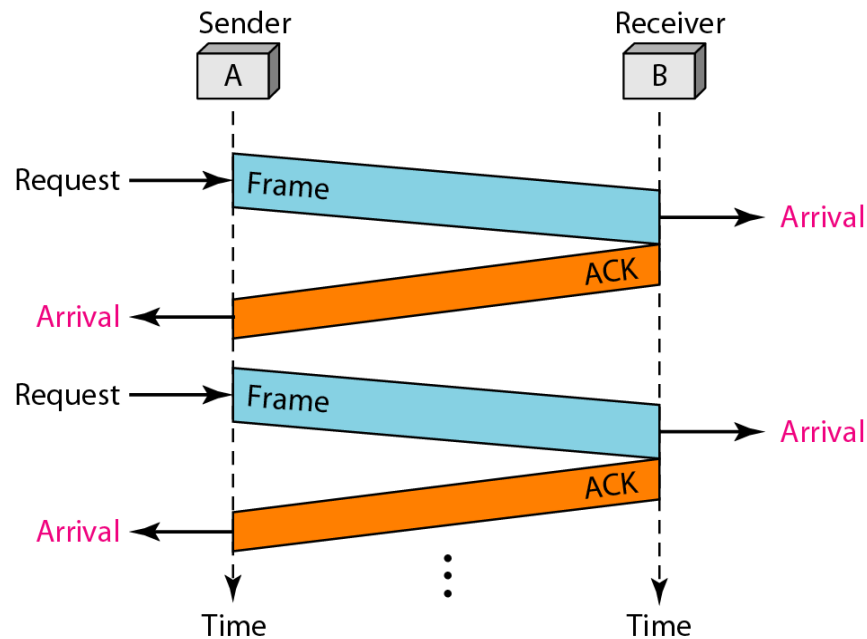
```
void receiver2(void)  
{  
    frame r, s;              /* buffers for frames */  
    event_type event;        /* frame_arrival is the only possibility */  
    while (true) {  
        wait_for_event(&event); /* only possibility is frame_arrival */  
        from_physical_layer(&r); /* go get the inbound frame */  
        to_network_layer(&r.info); /* pass the data to the network layer */  
        to_physical_layer(&s);    /* send a dummy frame to awaken sender */  
    }  
}
```

Stop-and-Wait Protocol

Simple tokens of ACK and flow control added



Stop-and-Wait Protocol: Example



A simplex stop-and-wait protocol:

- *Receiver 2 is that after delivering a packet to the network layer, receiver 2 sends an **acknowledgement frame** back to the sender before entering the wait loop again. because the only arrival of the frame back at the sender is important. the receiver need not put any particular information on it.*
- *points:*
 - *1) sender start out fetching a packet from the network layer, using it to construct a frame and sending it on its way.*
 - *2) sender must wait until an acknowledgement frame arrives before looping back and fetching the next packet from the network layer*

A simplex protocol for a Noisy Channel

- *The channel is noisy, frames may be damaged or lost*
- *Good scene : data frame reaches intact, ack sent back and received, next frame sent*
- *Bad scene :*
 - *Data frame damaged or lost ..hence no ack – sender times out and resends .. No problems*
 - *Data frame reaches intact but Ack lost .. Times out ..resends.. Receiver receives duplicate frames. Problem*
 - *Solution :Keep a sequence number for each frame to distinguish between the new frame and a duplicate frame*

A simplex protocol for a Noisy Channel

- *What should be minimum number of bits required for the sequence number?*
 - *The only ambiguity in this protocol is between a frame, m , and its direct successor, $m+1$. If a frame m is lost or damaged, the receiver will not acknowledge it, so the sender will keep trying to send it. Once it has been correctly received, the receiver will send an ack back to the sender. Depending upon whether the acknowledgement frame gets back to the sender correctly or not, the sender may try to send m or $m+1$. The event triggers the sender to start sending $m+2$ is the arrival of an acknowledgement $m+1$.*
 - *Protocols in which the sender waits for a positive acknowledgement before advancing to the next data item are often called (PAR-Positive acknowledgement retransmission or ARQ- Automatic Repeat Request)*

A simplex protocol for a Noisy Channel

```
/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */

#define MAX_SEQ 1 /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
    seq_nr next_frame_to_send; /* seq number of next outgoing frame */
    frame s; /* scratch variable */
    packet buffer; /* buffer for an outbound packet */
    event_type event;

    next_frame_to_send = 0; /* initialize outbound sequence numbers */
    from_network_layer(&buffer); /* fetch first packet */
    while (true) {
        s.info = buffer; /* construct a frame for transmission */
        s.seq = next_frame_to_send; /* insert sequence number in frame */
        to_physical_layer(&s); /* send it on its way */
        start_timer(s.seq); /* if answer takes too long, time out */
        wait_for_event(&event); /* frame_arrival, cksum_err, timeout */
        if (event == frame_arrival) {
            from_physical_layer(&s); /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                stop_timer(s.ack); /* turn the timer off */
                from_network_layer(&buffer); /* get the next one to send */
                inc(next_frame_to_send); /* invert next_frame_to_send */
            }
        }
    }
}
```

*A positive
acknowledg
ement with
retransmissi
on protocol.*

A simplex protocol for a Noisy Channel

```
void receiver3(void)
{
    seq_nr frame_expected;
    frame r, s;
    event_type event;

    frame_expected = 0;
    while (true) {
        wait_for_event(&event);           /* possibilities: frame_arrival, cksum_err */
        if (event == frame_arrival) {     /* a valid frame has arrived. */
            from_physical_layer(&r);      /* go get the newly arrived frame */
            if (r.seq == frame_expected) { /* this is what we have been waiting for. */
                to_network_layer(&r.info); /* pass the data to the network layer */
                inc(frame_expected);      /* next time expect the other sequence nr */
            }
            s.ack = 1 - frame_expected;    /* tell which frame is being acked */
            to_physical_layer(&s);        /* send acknowledgement */
        }
    }
}
```

A positive acknowledgement with retransmission protocol.

Noisy Channels: Stop-and-Wait ARQ

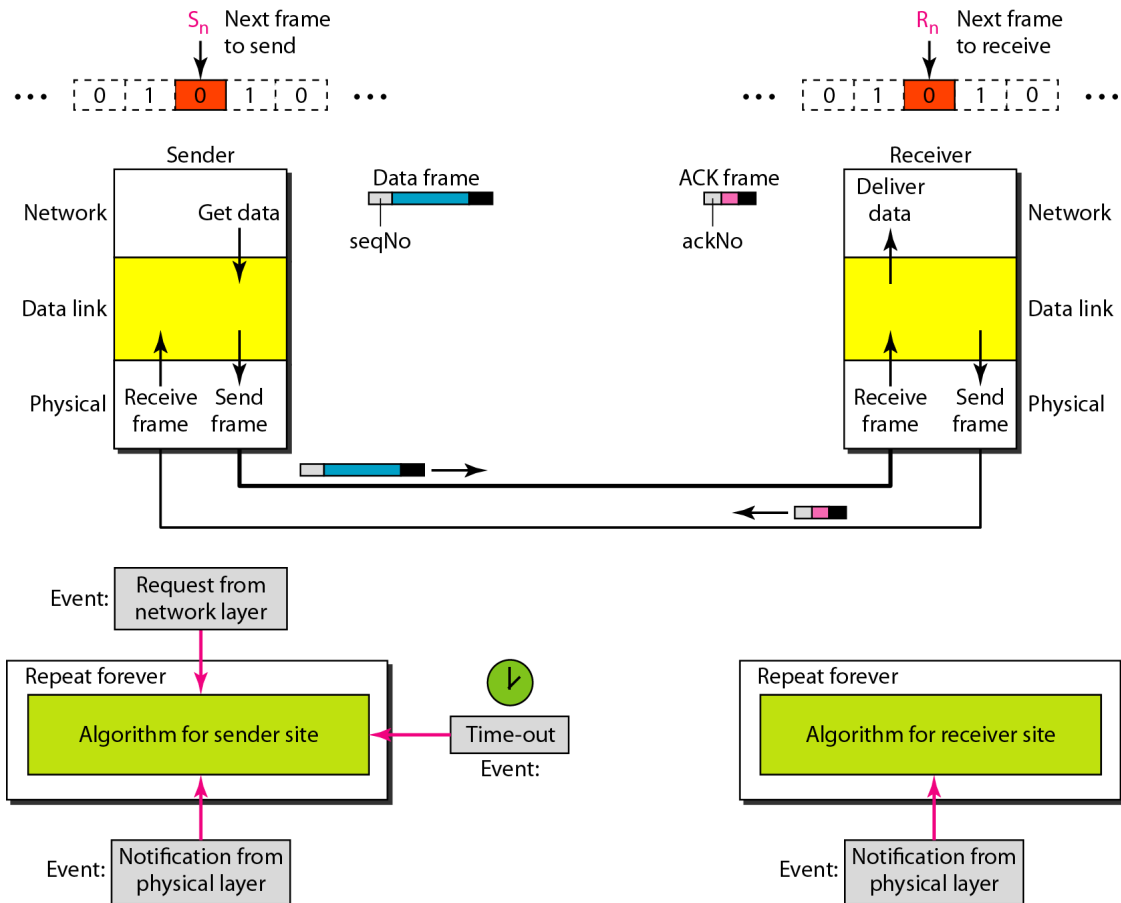
Stop-and-wait Automatic Repeat Request (ARQ)

Error correction in Stop-and-Wait ARQ is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires

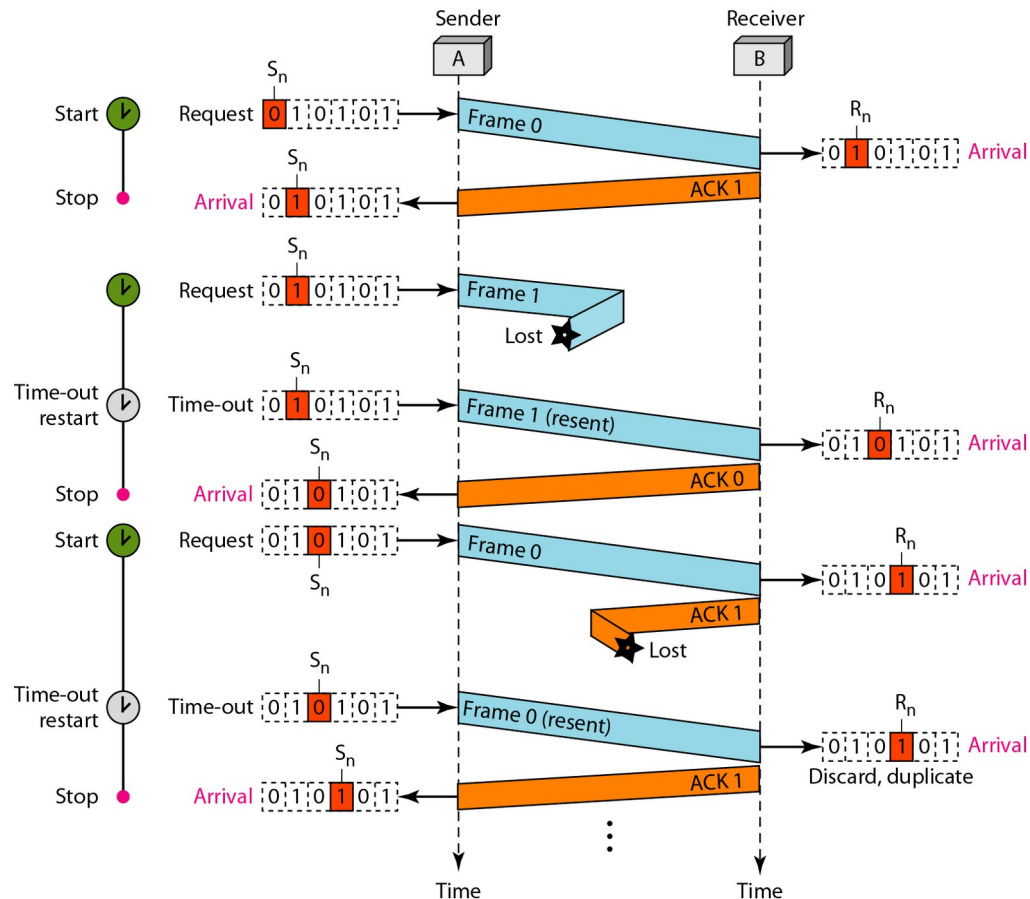
In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic

Acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected.

Stop-and-Wait ARQ



Stop-and-Wait ARQ: Example



Contd..

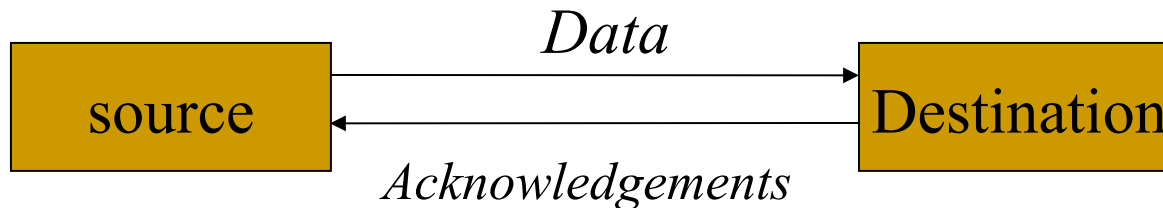
- The Stop-and-Wait ARQ is very inefficient if our channel has a large bandwidth and long round-trip delay.
 - There is no Pipelining in this protocol because we need to wait for a frame to reach the destination and be acknowledged before the next frame is sent.
-

Sliding window protocols

- *Must be able to transmit data in both directions.*
- *Choices for utilization of the reverse channel:*
 - *mix DATA frames with ACK frames.*
 - *Piggyback the ACK*
 - *Receiver waits for DATA traffic in the opposite direction.*
 - *Use the ACK field in the frame header to send sequence number of frame being ACKed.*
 - *➔ better use of the channel capacity.*

Sliding window protocols:

- *In the Previous protocols ,Data frames were transmitted in one direction only.*
- *In most practical situations ,there is a need for transmitting data in both directions.*
- *One way of achieving full-duplex data transmission is to have two separate communication channels and each one for simplex data traffic (in different directions)*
- *we have two separate physical circuits ,each with a “forward” channel (for data) and a “reverse” channel(for acknowledgements).*

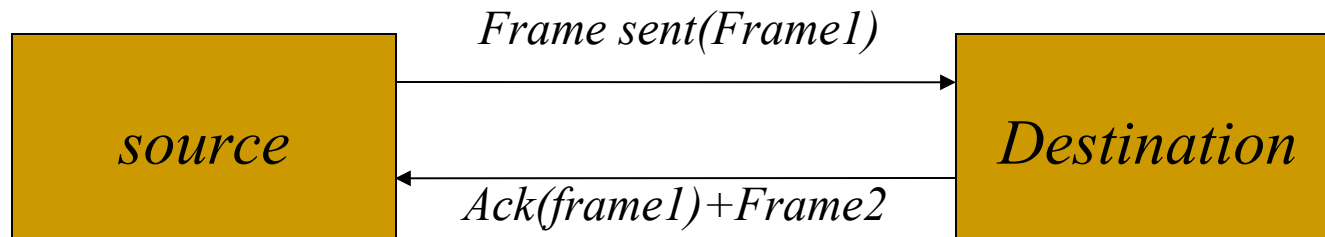


Sliding window protocols:

- *Disadvantage: The bandwidth of the reverse channel is almost entirely wasted.*
- *A better idea is to use the same circuits for data in both directions*
- *In this model the data frames from A and B are intermixed with the acknowledgement frames from A to B.*
- *Kind: kind field in the header of an incoming frame, the receiver can tell whether the frame is data or acknowledgements.*

Sliding window protocols:

■ Piggybacking:



- When a data frame arrives, instead of immediately sending a separate control frame, the receiver restrains itself and waits until the network layer passes it the next packet.
- The *acknowledgement* is attached to the outgoing data frame
- *Disadv*: This technique is temporarily delaying outgoing acknowledgements.
- If the datalinklayer waits *longer than the senders timeout period*, the frame will be retransmitted.

Sliding window protocols:

- ***Rule:** sender waiting a fixed number of milliseconds. If a new packet arrives quickly the acknowledgement is piggybacked onto it. otherwise if no new packet has arrived by the end of this time period ,the data link layer just sends a separate acknowledgement frame.*
- *In sliding window protocol each frame contains a sequence number ranging from 0 up to some maximum.*
- *The maximum is usually 2^n-1 so the sequence number fits nicely in an n -bit field.*
- *The stop-and-wait sliding window protocol uses $n=1$ restricting the sequence numbers 0 and 1.*
- *The sender must keep all these frames in its memory for possible retransmission*

Sliding window protocols:

- *Thus if the maximum window size is n , the sender needs n buffers to hold the unacknowledged frames*

- *3 bit field -000*

001

010..etc

here $n=3$ i.e. $2^3-1=7$

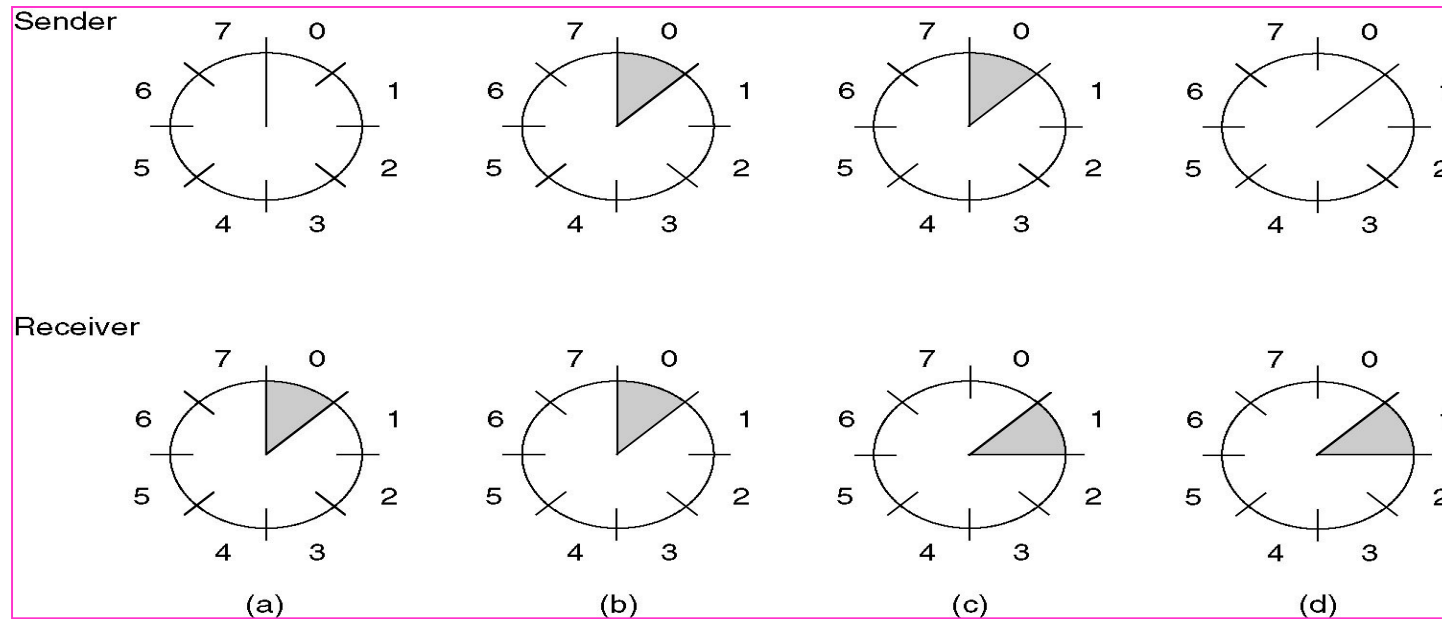
window size is 0 to 7

Sliding window :: sender has a window of frames and maintains a list of consecutive sequence numbers for frames that it is permitted to send without waiting for ACKs.

receiver has a window that is a list of frame sequence numbers it is permitted to accept.

Note – sending and receiving windows do NOT have to be the same size.

Sliding window protocols:



A sliding window of size 1, with a 3-bit sequence number.

(a) Initially.

(b) After the first frame has been sent.

(c) After the first frame has been received.

(d) After the first acknowledgement has been received.

Sliding window protocols:

■ *Sliding window protocols: 3 methods*

- ❑ *1) 1-bit sliding window protocol*
- ❑ *2) Go –Back N*
- ❑ *3) Selective Repeat*

■ *1-bit sliding window protocol:*

- ❑ *Window size 1.*
- ❑ *Stop-and-wait.*
- ❑ *Must get ack before can send next frame.*
- ❑ *Both machines are sending and receiving.*

1-bit sliding window protocol

```
/* Protocol 4 (sliding window) is bidirectional. */
#define MAX_SEQ 1 /* must be 1 for protocol 4 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"
void protocol4 (void)
{
    seq_nr next_frame_to_send; /* 0 or 1 only */
    seq_nr frame_expected; /* 0 or 1 only */
    frame r, s; /* scratch variables */
    packet buffer; /* current packet being sent */
    event_type event;

    next_frame_to_send = 0; /* next frame on the outbound stream */
    frame_expected = 0; /* frame expected next */
    from_network_layer(&buffer); /* fetch a packet from the network layer */
    s.info = buffer; /* prepare to send the initial frame */
    s.seq = next_frame_to_send; /* insert sequence number into frame */
    s.ack = 1 - frame_expected; /* piggybacked ack */
    to_physical_layer(&s); /* transmit the frame */
    start_timer(s.seq); /* start the timer running */
}
```

1-bit sliding window protocol

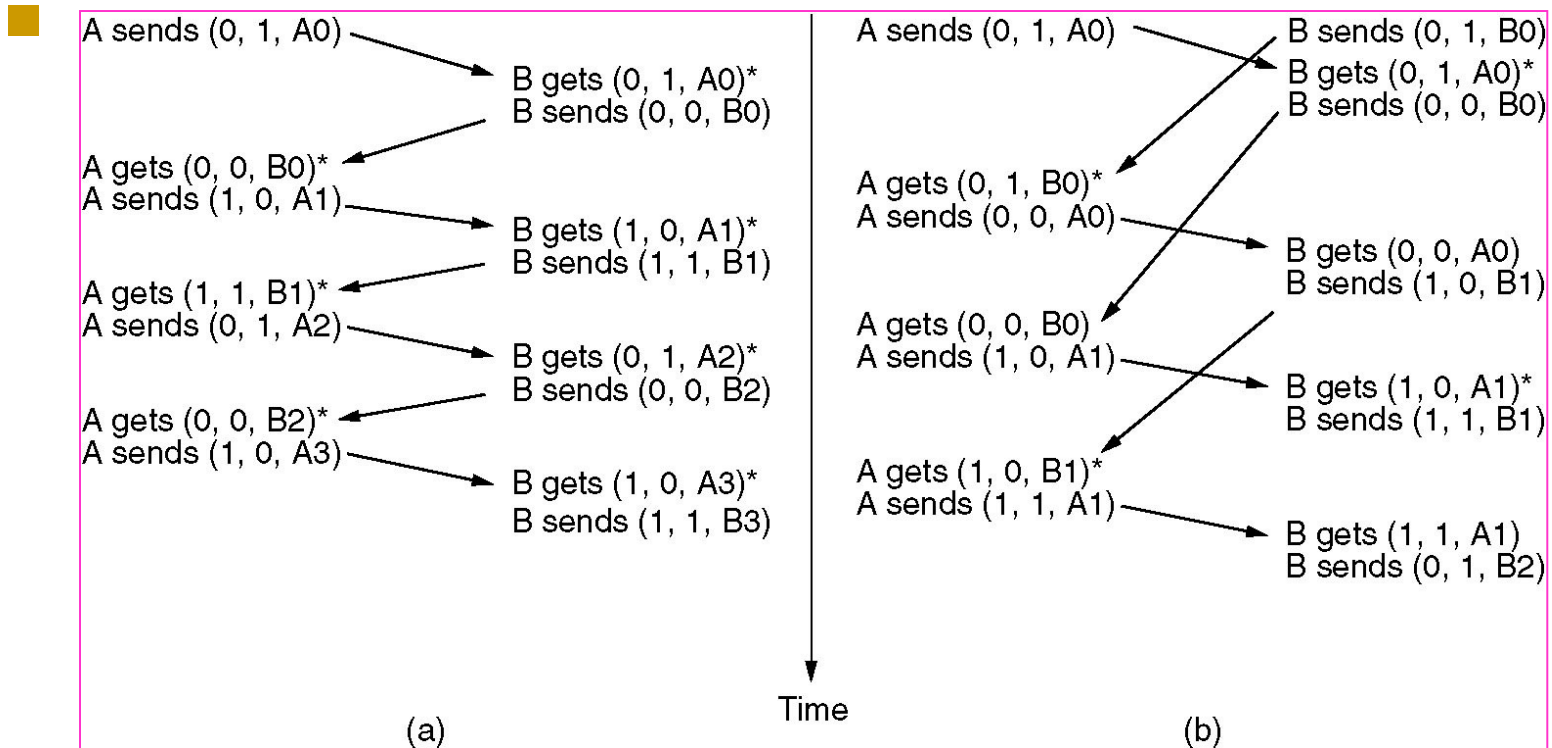
```
while (true) {  
    wait_for_event(&event);           /* frame_arrival, cksum_err, or timeout */  
    if (event == frame_arrival) {     /* a frame has arrived undamaged. */  
        from_physical_layer(&r);      /* go get it */  
  
        if (r.seq == frame_expected) { /* handle inbound frame stream. */  
            to_network_layer(&r.info); /* pass packet to network layer */  
            inc(frame_expected);       /* invert seq number expected next */  
        }  
  
        if (r.ack == next_frame_to_send) { /* handle outbound frame stream. */  
            stop_timer(r.ack);          /* turn the timer off */  
            from_network_layer(&buffer); /* fetch new pkt from network layer */  
            inc(next_frame_to_send);    /* invert sender's sequence number */  
        }  
    }  
    s.info = buffer;                  /* construct outbound frame */  
    s.seq = next_frame_to_send;       /* insert sequence number into it */  
    s.ack = 1 - frame_expected;       /* seq number of last received frame */  
    to_physical_layer(&s);            /* transmit a frame */  
    start_timer(s.seq);               /* start the timer running */  
}  
}
```

1-bit sliding protocol :

■ Example:

- *A trying to send its frame 0 to B.
B trying to send its frame 0 to A.*
- *Imagine A's timeout is too short. A repeatedly times out and sends multiple copies to B, all with $seq=0$, $ack=1$.
When first one of these gets to B, it is accepted. Set $expected=1$. B sends its frame, $seq=0$, $ack=0$.
All subsequent copies of A's frame rejected since $seq=0$ not equal to $expected$. All these also have $ack=1$.
B repeatedly sends its frame, $seq=0$, $ack=0$. But A not getting it because it is timing out too soon.
Eventually, A gets one of these frames. A has its ack now (and B's frame). A sends next frame and acks B's frame.*

1-bit sliding protocol :



Two scenarios for protocol 4. (a) Normal case. (b) Abnormal case. The notation is (seq, ack, packet number). An asterisk indicates where a network layer accepts a packet.

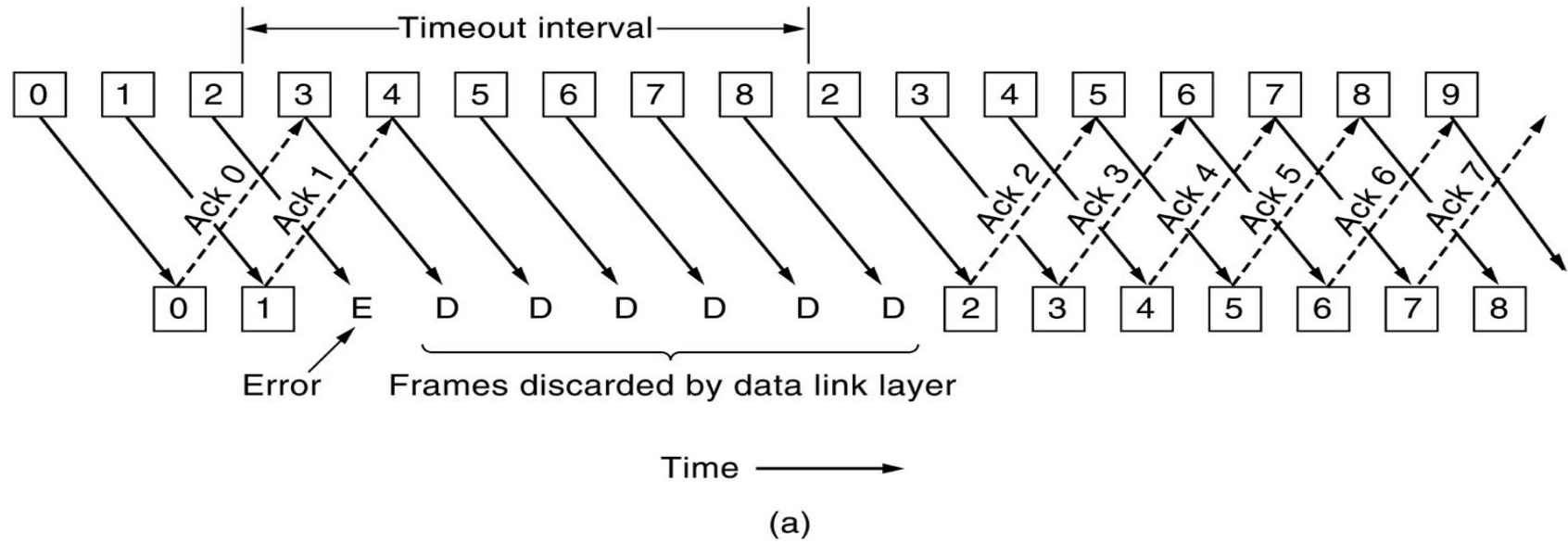
A protocol using Go Back N:

Go Back n, is for the receiver simply to discard all subsequent frames, sending no acknowledgements for the discarded frames. This strategy corresponds to a receive window of size 1.

The data link layer refuses to accept any frame except the next one it must give to the network layer.

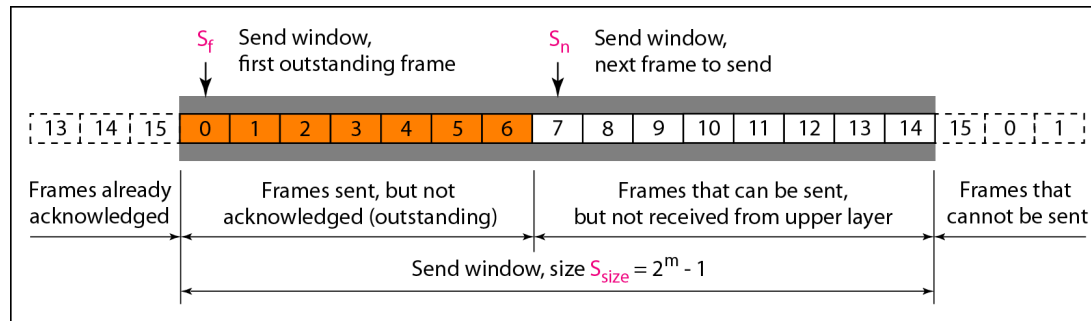
Points: fig (a)

- 1) frame 0 and 1 correctly received and Acknowledged*
- 2) Frame 2 is damaged or lost ,The sender continuous to send frames until the timer for frame 2 expires.*
- 3) Then it backs up to frame 2 and starts all over with it ,sending 2,3,4 etc. all over again.*

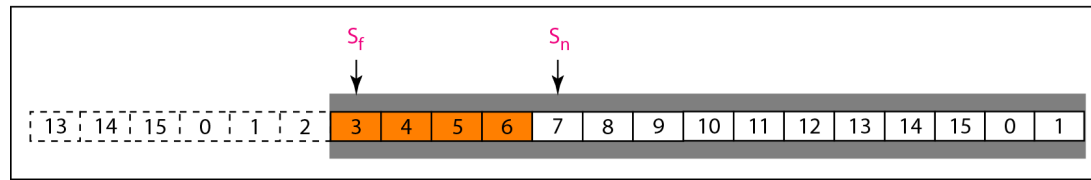


Go-Back-N ARQ

- Pipelining improves the efficiency of the transmission
- In the Go-Back-N Protocol, the sequence numbers are modulo 2^m , where m is the size of the sequence number field in bits
- The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: S_f , S_n , and S_{size}
- The send window can slide one or more slots when a valid acknowledgment arrives.



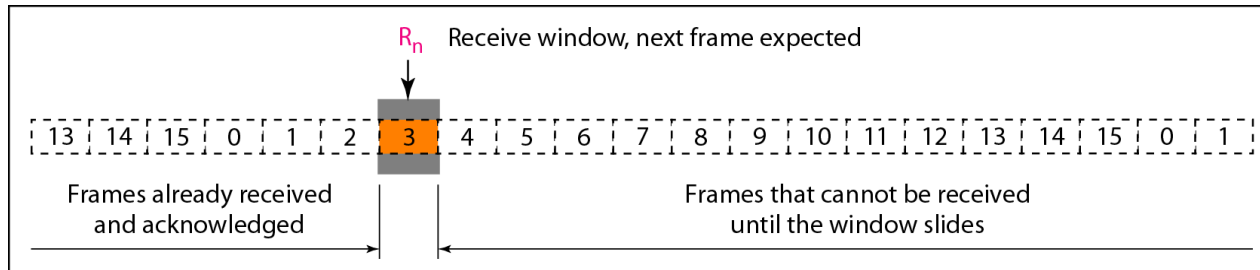
a. Send window before sliding



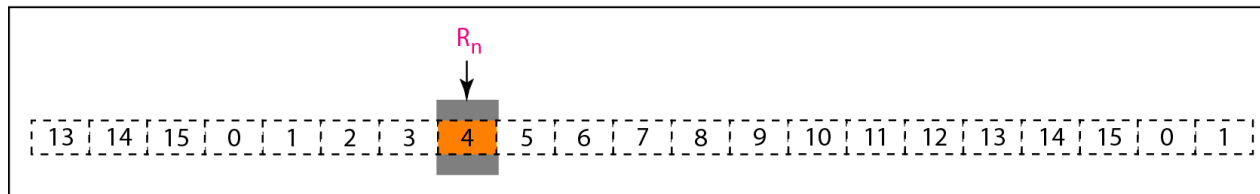
b. Send window after sliding

Go-Back-N ARQ

- Receive window for Go-Back-N ARQ
- The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n . The window slides when a correct frame has arrived; sliding occurs one slot at a time.



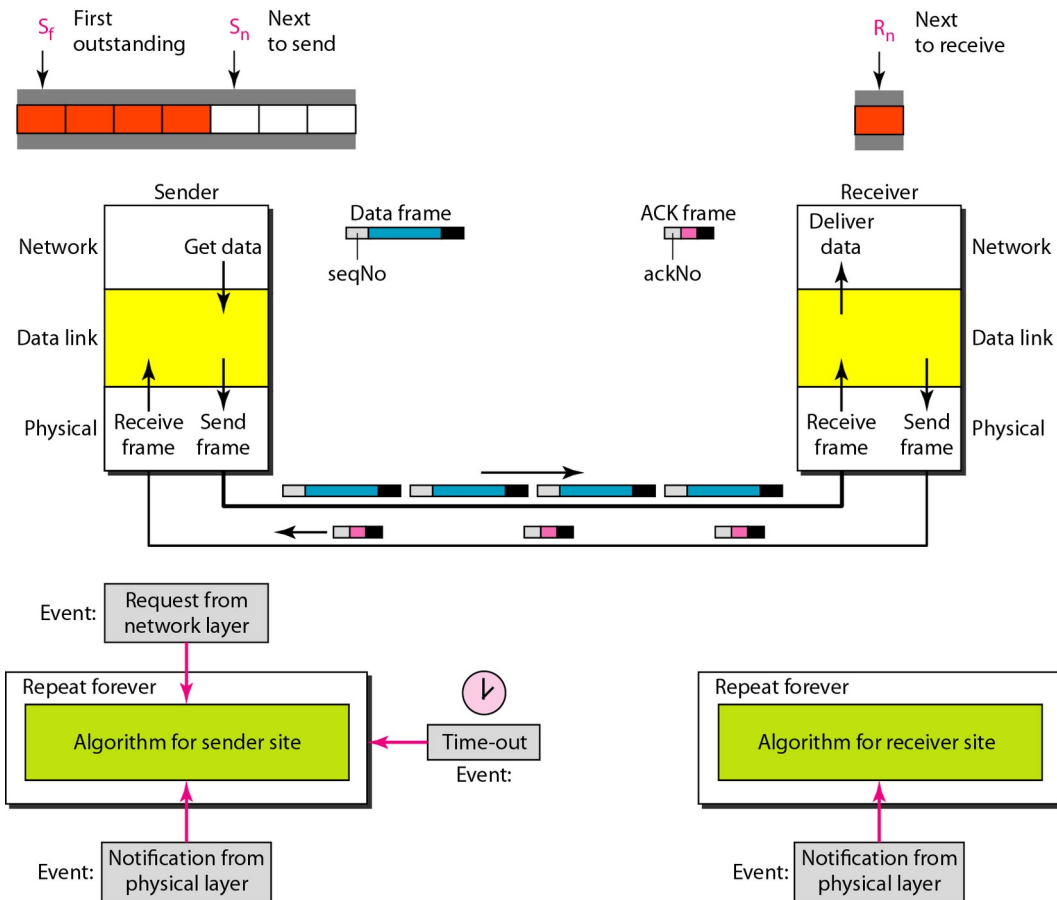
a. Receive window



b. Window after sliding

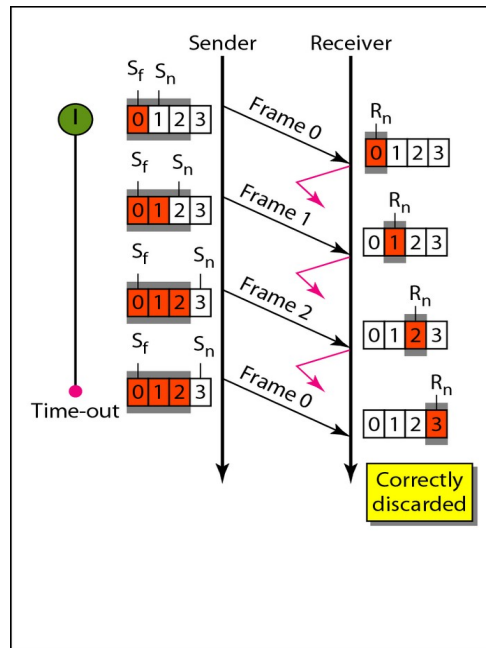
Go-Back-N ARQ

- Sliding windows, Timers, ACK, Resending a frame

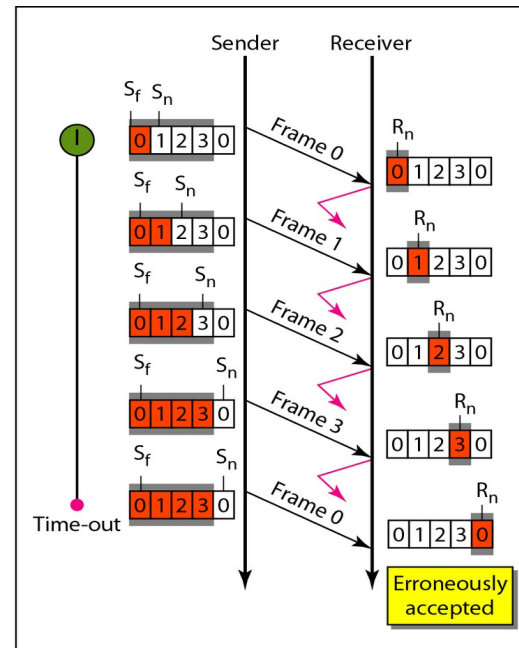


Go-Back-N ARQ: Send Window Size

- In Go-Back-N ARQ, the size of the send window must be less than 2^m ; the size of the receiver window is always 1
- Stop-and-Wait ARQ is a special case of Go-Back-N ARQ in which the size of the send window is 1



a. Window size $< 2^m$



b. Window size $= 2^m$

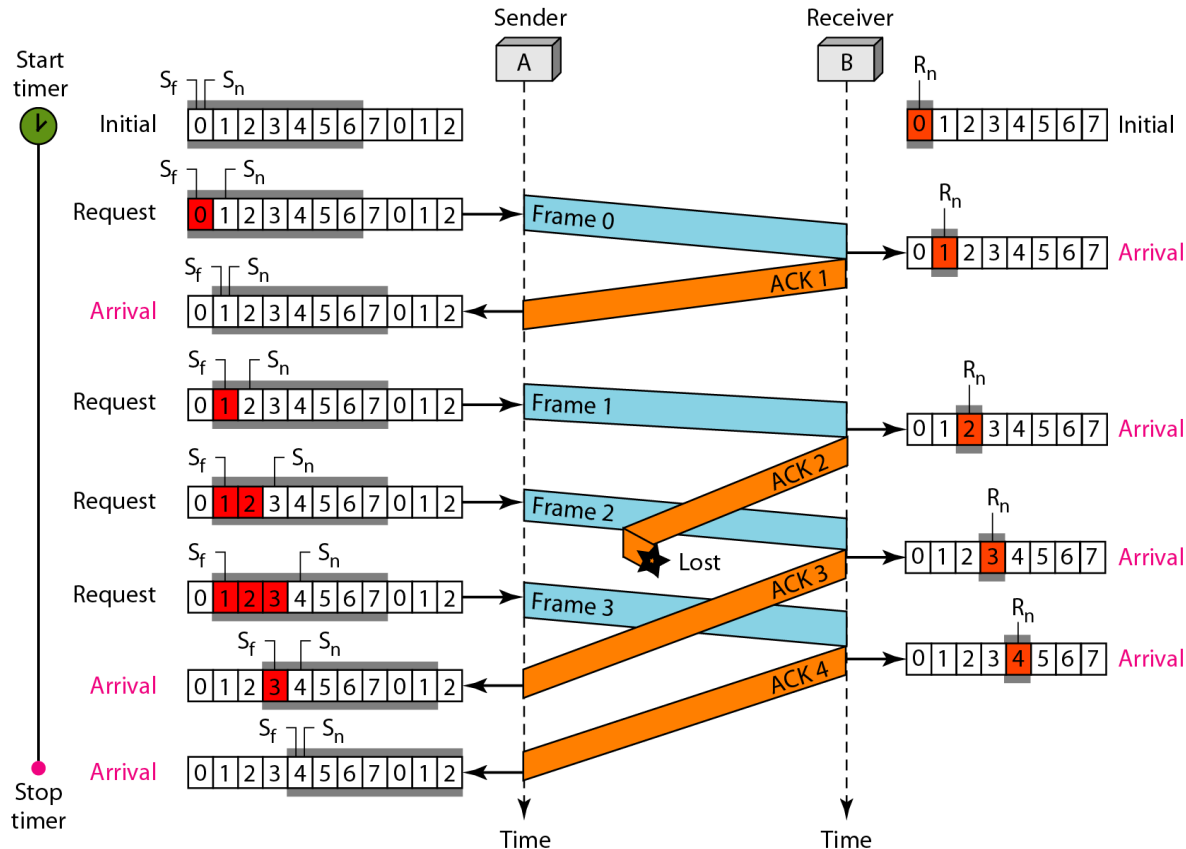
Go-Back-N ARQ: Sender Algorithm

```
1  $S_w = 2^m - 1;$ 
2  $S_f = 0;$ 
3  $S_n = 0;$ 
4
5 while (true)                                //Repeat forever
6 {
7   WaitForEvent();
8   if(Event(RequestToSend))                  //A packet to send
9   {
10     if( $S_n - S_f \geq S_w$ )                     //If window is full
11       Sleep();
12     GetData();
13     MakeFrame( $S_n$ );
14     StoreFrame( $S_n$ );
15     SendFrame( $S_n$ );
16      $S_n = S_n + 1;$ 
17     if(timer not running)
18       StartTimer();
19   }
20
21   if(Event(ArrivalNotification)) //ACK arrives
22   {
23     Receive(ACK);
24     if(corrupted(ACK))
25       Sleep();
26     if( $(ackNo > S_f) \&\& (ackNo \leq S_n)$ ) //If a valid ACK
27       While( $S_f \leq ackNo$ )
28       {
29         PurgeFrame( $S_f$ );
30          $S_f = S_f + 1;$ 
31       }
32       StopTimer();
33   }
34
35   if(Event(TimeOut))                        //The timer expires
36   {
37     StartTimer();
38     Temp =  $S_f$ ;
39     while(Temp <  $S_n$ );
40     {
41       SendFrame( $S_f$ );
42        $S_f = S_f + 1;$ 
43     }
44   }
45 }
```

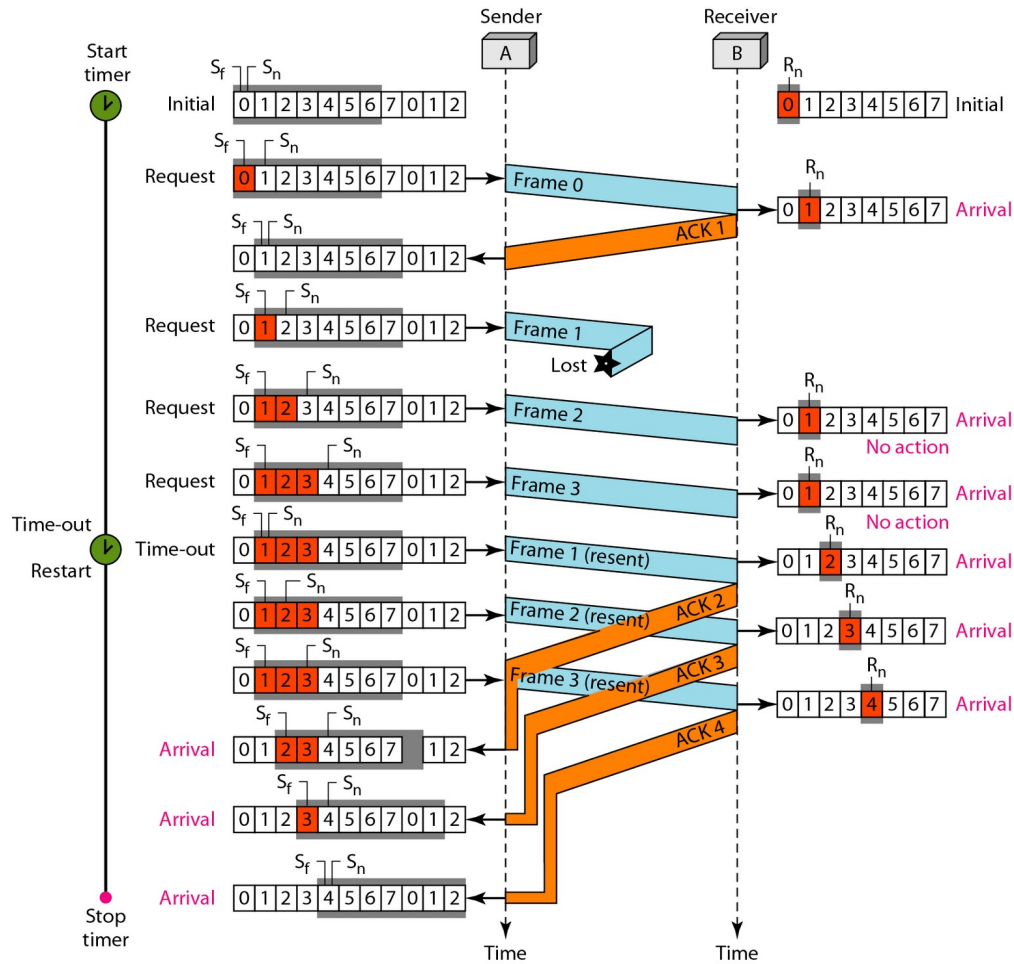

Go-Back-N ARQ: Receiver Algorithm

```
1 Rn = 0;
2
3 while (true)                                //Repeat forever
4 {
5     WaitForEvent();
6
7     if(Event(ArrivalNotification)) //Data frame arrives
8     {
9         Receive(Frame);
10        if(corrupted(Frame))
11            Sleep();
12        if(seqNo == Rn)                //If expected frame
13        {
14            DeliverData();              //Deliver data
15            Rn = Rn + 1;                //Slide window
16            SendACK(Rn);
17        }
18    }
19 }
```

Go-Back-N ARQ: Example 1



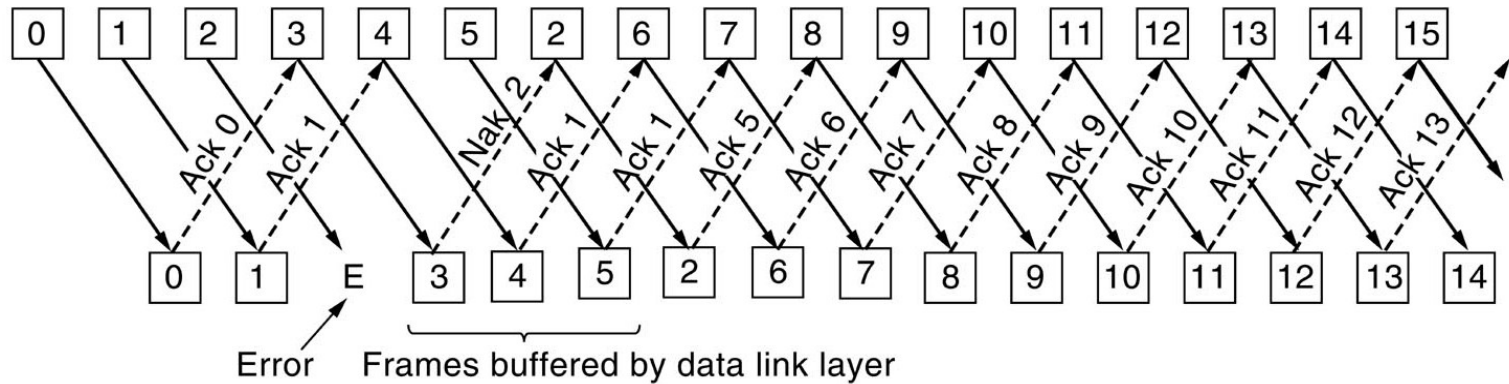
Go-Back-N ARQ: Example 2



Selective Repeat:

- *When selective repeat is used ,a bad frame that is received is discarded. But good frames after it are buffered.*
- *when the senders timeout ,only the oldest unacknowledged frame is retransmitted.*
- *If that frames arrives correctly ,the receiver can deliver it to the network layer in sequence all the frames it has buffered.*
- *fig (b) :*
 - *1) Frames 0 and 1 correctly received and acknowledged.*
 - *2) Frame 2 is lost, when frame 3 arrives at the receiver ,the data link layer notices that it has a missed frame. so it sends back a NAK for 2 but buffers 3*
 - *3) when frame 4 and 5 arrive ,they are buffered by the data link layer instead of being passed to the network layer*
 - *4)The NAK 2 gets back to the sender which immediately resends frame 2.*

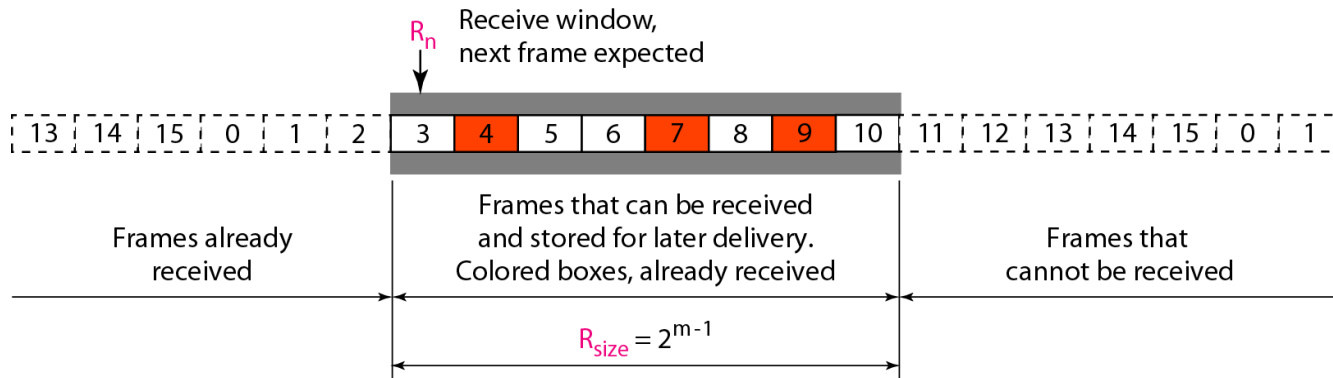
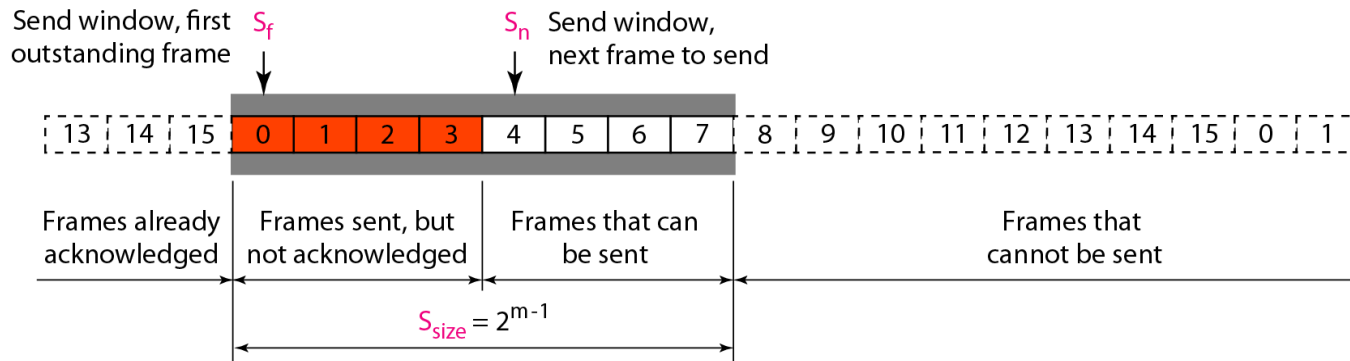
A protocol using selective repeat:



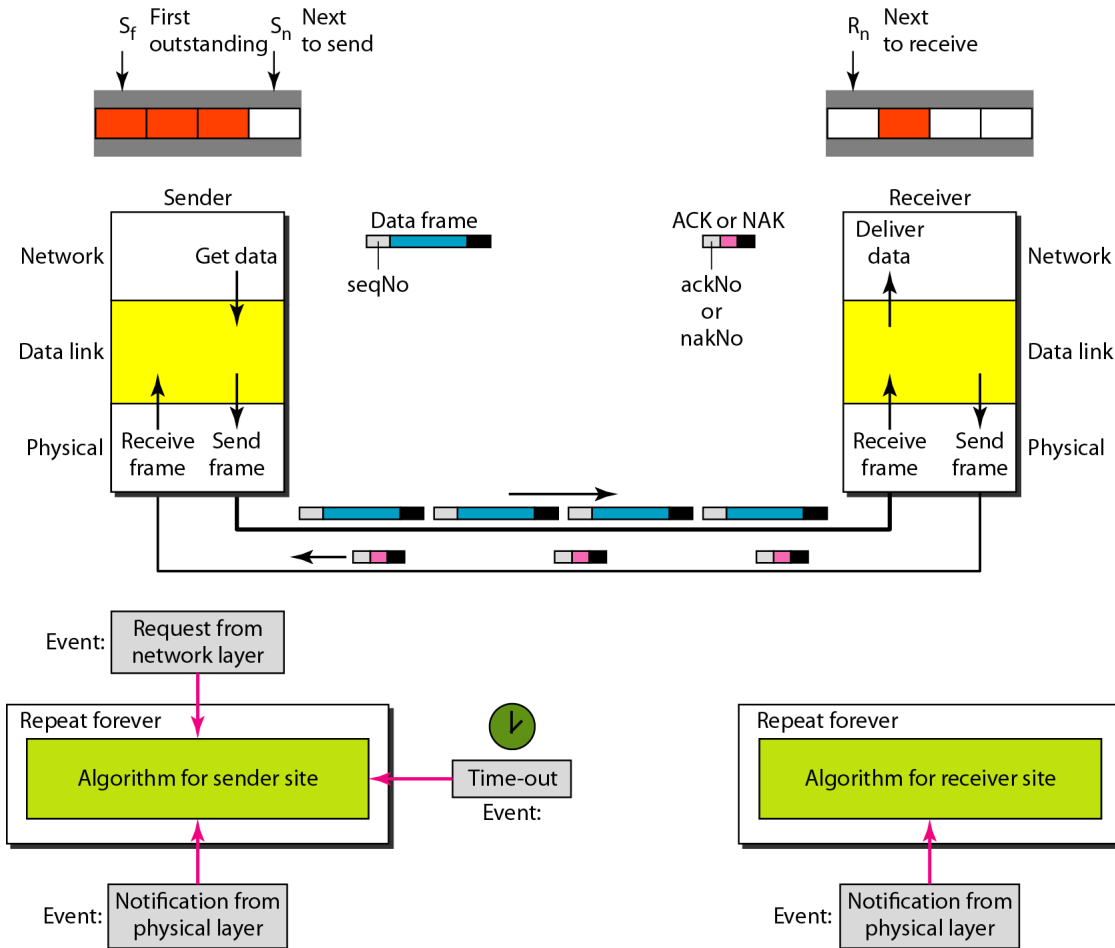
(b)

Selective Repeat ARQ

- Sender window size

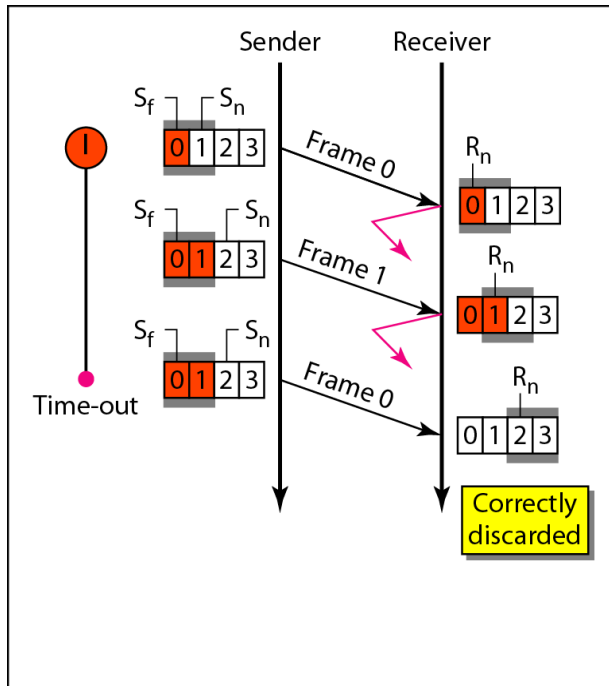


Selective Repeat ARQ

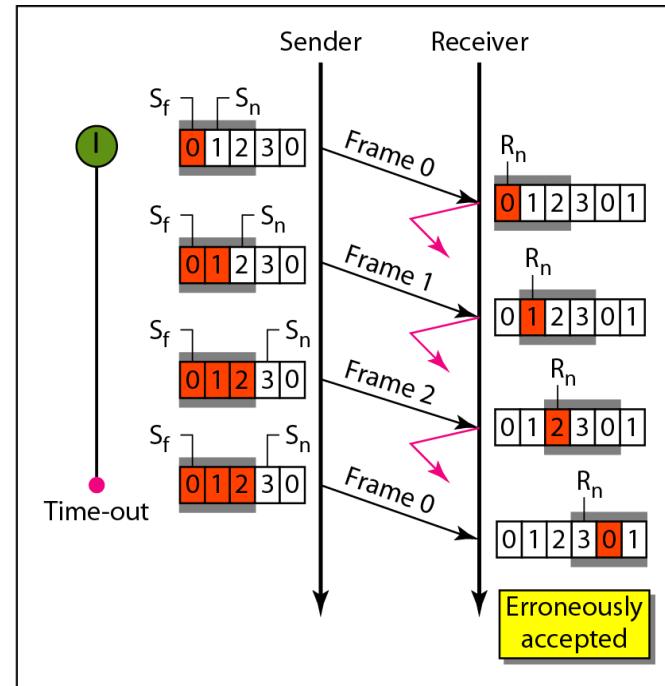


Selective Repeat ARQ: Window Size

- The size of the sender and receiver window must be at most one-half of 2^m



a. Window size = 2^{m-1}



b. Window size $> 2^{m-1}$

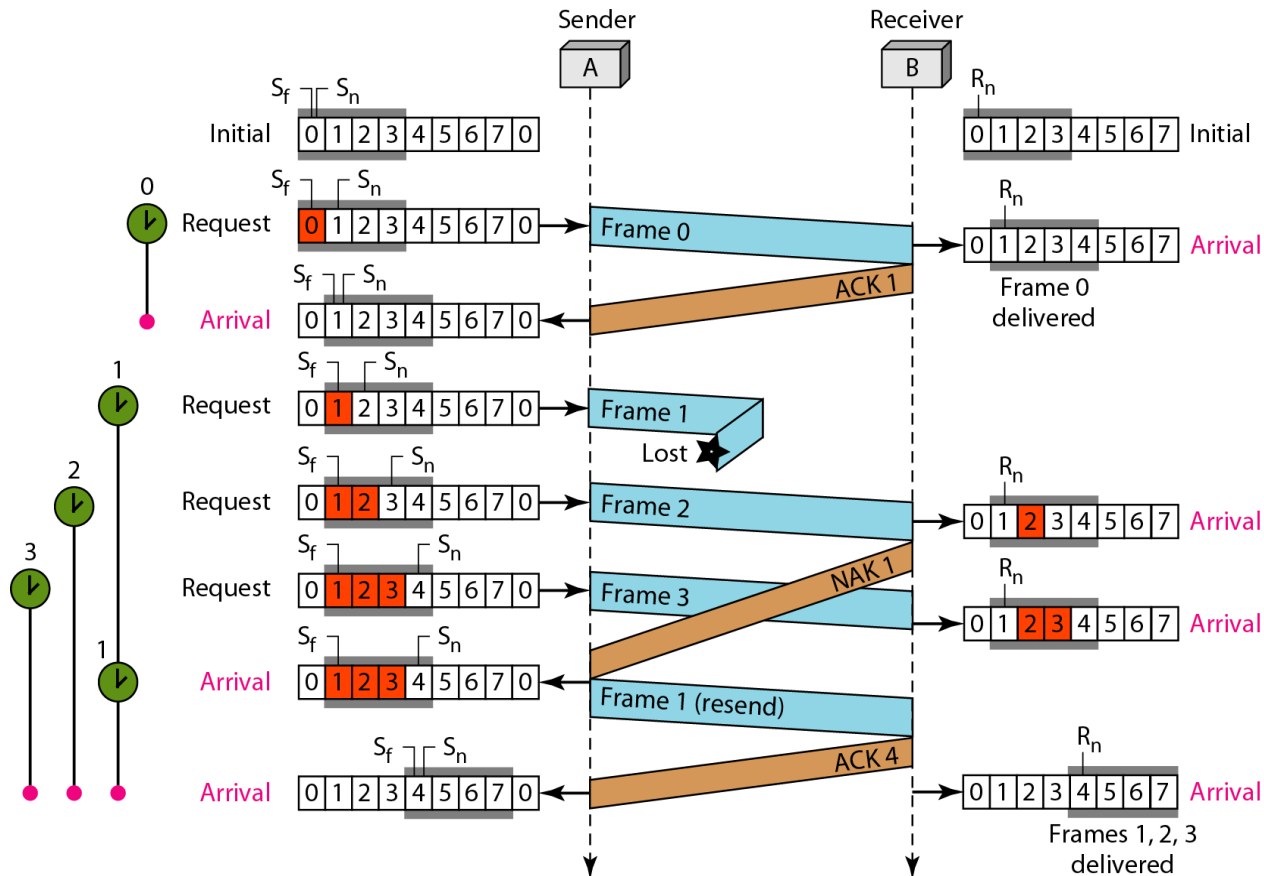
Selective Repeat ARQ: Sender-Site Algorithm

```
10     if( $S_n - S_f \geq S_w$ ) //If window is full
11         Sleep();
12     GetData();
13     MakeFrame( $S_n$ );
14     StoreFrame( $S_n$ );
15     SendFrame( $S_n$ );
16      $S_n = S_n + 1$ ;
17     StartTimer( $S_n$ );
18 }
20 if(Event(ArrivalNotification)) //ACK arrives
21 {
22     Receive(frame); //Receive ACK or NAK
23     if(corrupted(frame))
24         Sleep();
25     if (FrameType == NAK)
26         if (nakNo between  $S_f$  and  $S_n$ )
27         {
28             resend(nakNo);
29             StartTimer(nakNo);
30         }
31     if (FrameType == ACK)
32         if (ackNo between  $S_f$  and  $S_n$ )
33         {
34             while( $s_f < \text{ackNo}$ )
35             {
36                 Purge( $s_f$ );
37                 StopTimer( $s_f$ );
38                  $S_f = S_f + 1$ ;
39             }
40         }
41 }
```

Selective Repeat ARQ: Receiver-Site Algorithm

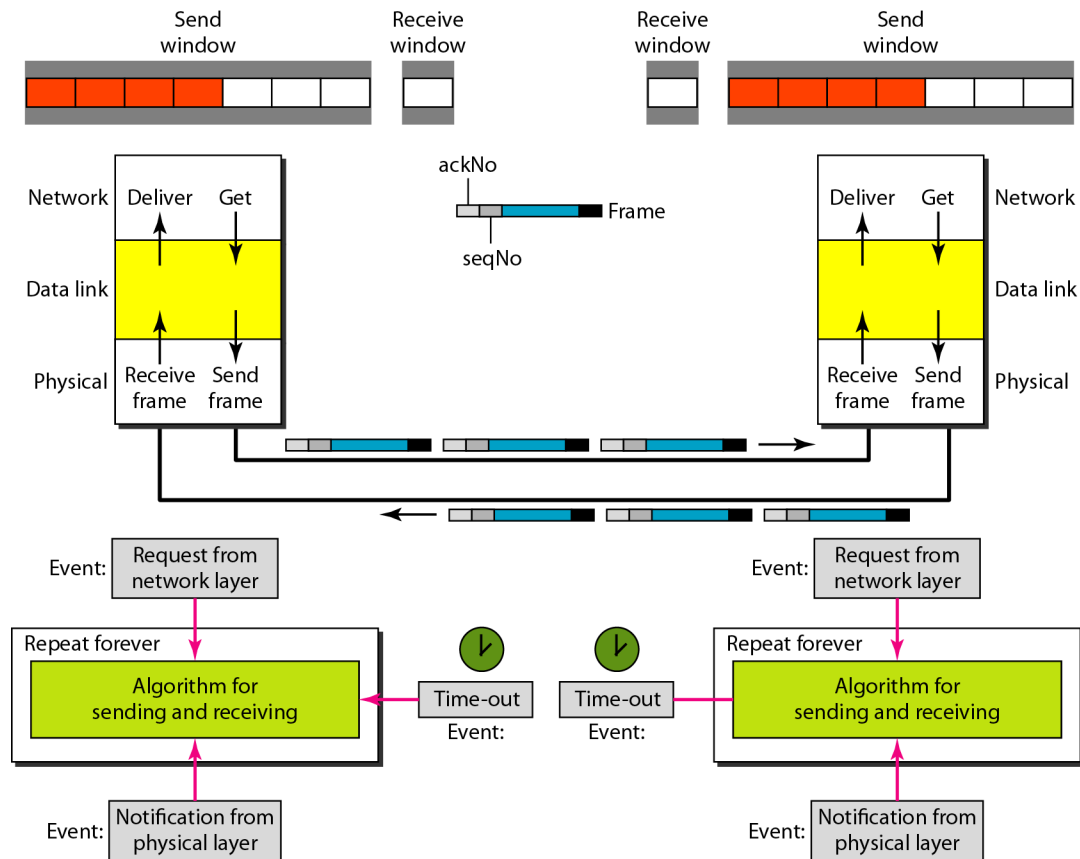
```
1  Rn = 0;
2  NakSent = false;
3  AckNeeded = false;
4  Repeat(for all slots)
5      Marked(slot) = false;
6
7  while (true)                                //Repeat forever
8  {
9      WaitForEvent();
10
11     if(Event(ArrivalNotification))           //Data frame arrives
12     {
13         Receive(Frame);
14         if(corrupted(Frame))&& (NOT NakSent)
15         {
16             SendNAK(Rn);
17             NakSent = true;
18             Sleep();
19         }
20         if(seqNo <> Rn)&& (NOT NakSent)
21         {
22             SendNAK(Rn);
23             NakSent = true;
24             if ((seqNo in window)&&(!Marked(seqNo)))
25             {
26                 StoreFrame(seqNo)
27                 Marked(seqNo)= true;
28                 while(Marked(Rn))
29                 {
30                     DeliverData(Rn);
31                     Purge(Rn);
32                     Rn = Rn + 1;
33                     AckNeeded = true;
34                 }
35                 if(AckNeeded);
36                 {
37                     SendAck(Rn);
38                     AckNeeded = false;
39                     NakSent = false;
40                 }
41             }
42         }
43     }
44 }
```

Selective Repeat ARQ: Example



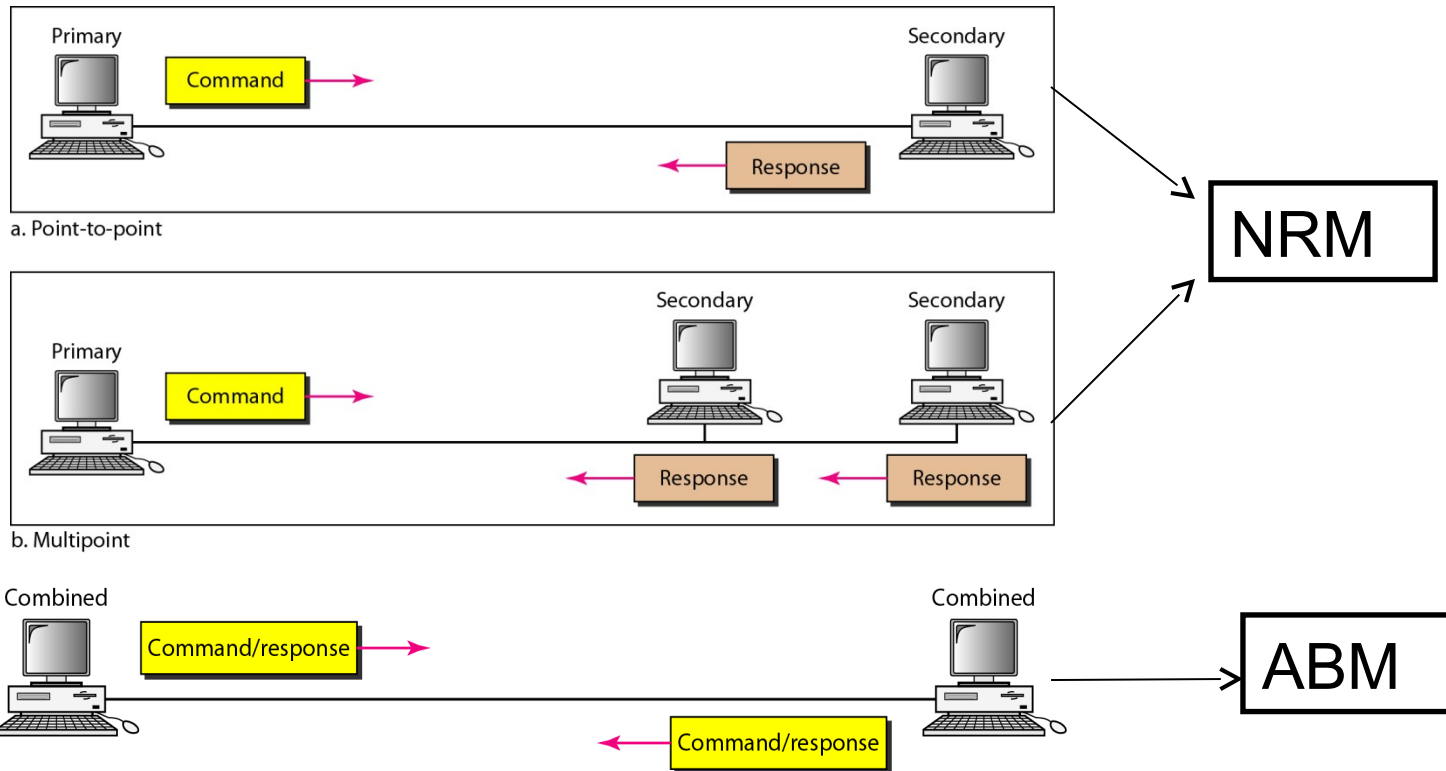
Piggybacking

- To improve the efficiency of the bidirectional protocols
- Piggybacking in Go-Back-N ARQ



HDLC

- High-level Data Link Control is a bit-oriented protocol for communication over point-to-point and multipoint links.
- Two common transfer mode: normal response mode (NRM) and asynchronous balanced mode (ABM)



FRAMES

- To provide the flexibility necessary to support all the options possible in the modes and configuration, HDLC defines three types of frames:

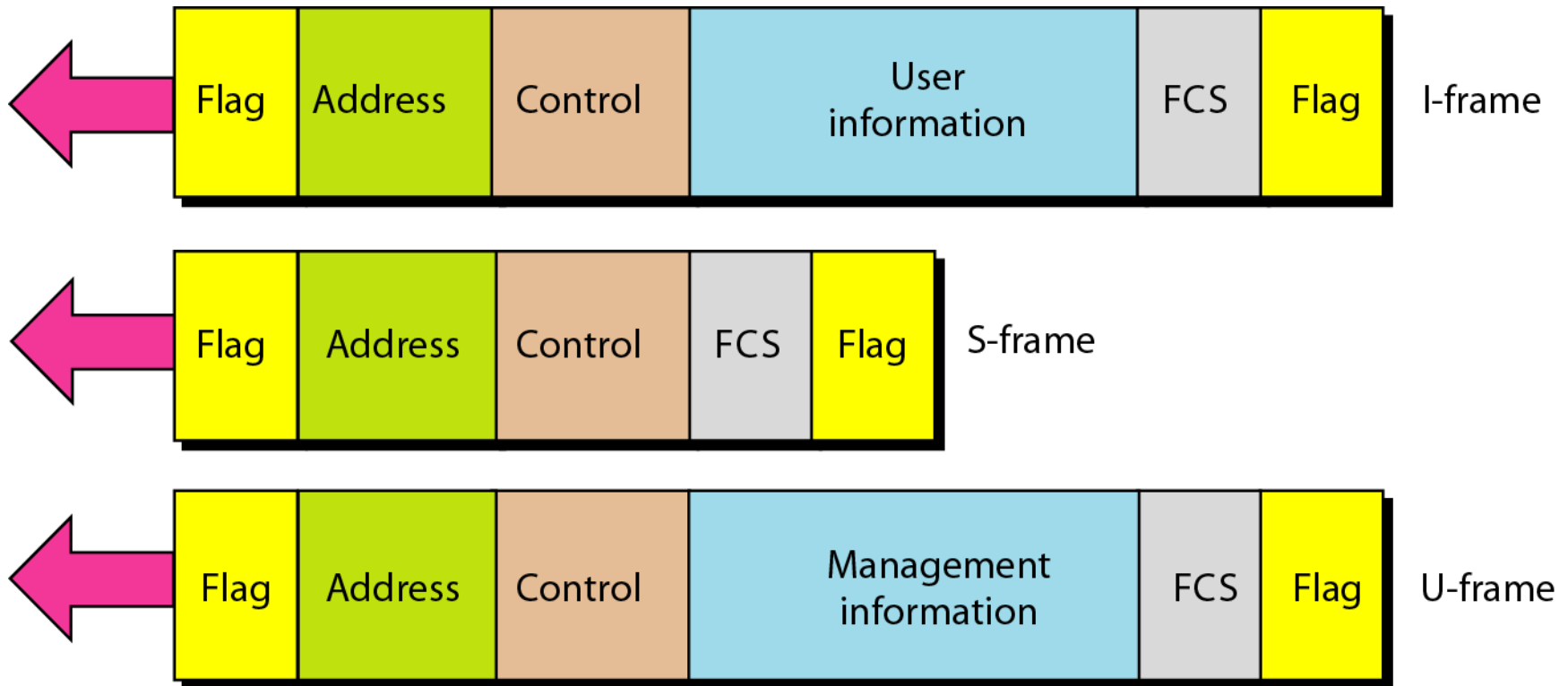
Information frames(I-frame)

Supervisory frames(S-frames)

Unnumbered frames(U-frames)

-
- I-frames are used to transport user data and the control information relating to user data(Piggybacking).
 - S-frames are used only to transport control information.
 - U-frames are reserved for system management. The information carried by U-frame is intended for managing the link itself.
-

HDLC frames:

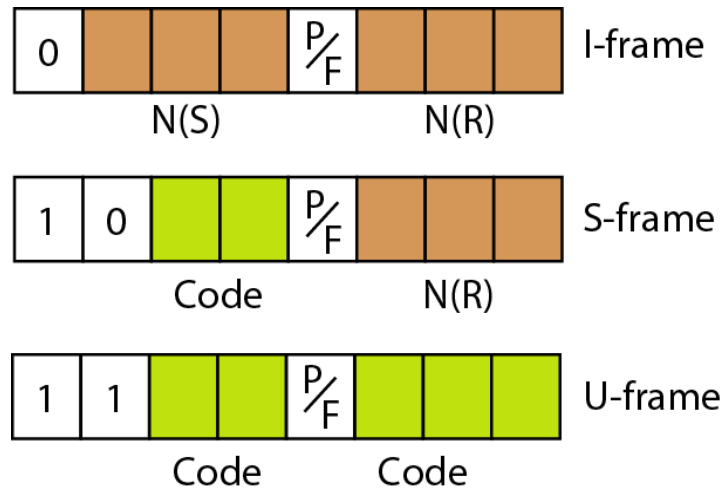


HDLC: Frames

- **Flag field:** 01111110 to identify both the beginning and the end of a frame and serve as synchronization pattern for receiver.
- **Address:** contains address of secondary station. If a primary station created the frame, it contains a *to* address. If secondary creates the frame, it contains a *from* address(1 byte or several bytes long).
- **Control field:** 1-or 2-bytes segment used for flow and error control (interpretation depends on the frame type).
- **Information field:** contains the user's data from the network layer or management information(its length vary from one network to another)
- **FCS field:** Forward Check Sequence is 2- or 4-byte ITU-T CRC for error detection

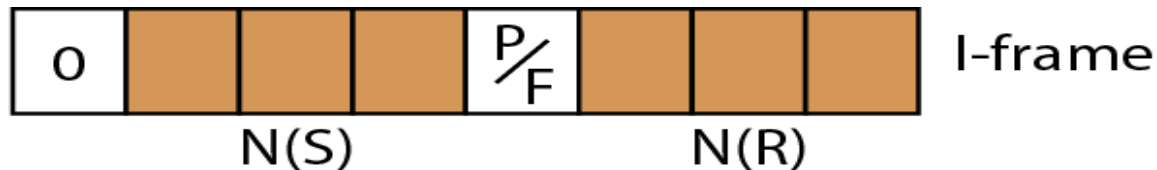
Control Field

- Control Field: 1- or 2-byte segment of the frame used for flow and error control
- Determine the type of frame and define its functionality
- Control field for I-frame: P/F (poll/final bit for primary/secondary)



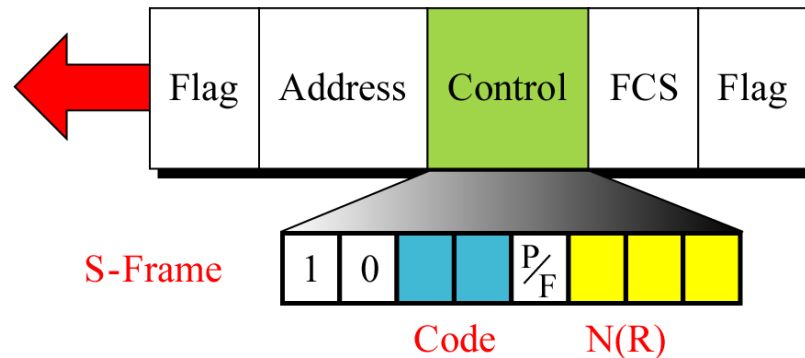
HDLC: Frames

- Control field for **I-frame**
- Carry user data from network layer.
- Include flow and error control
- The first bit defines the type. If the first bit of control field is 0, this means it is I-frame.
- N(S) defines a sequence number of the frame. As it is three bits we can define sequence no between 0 and 7
- N(R) correspond to the acknowledgment number when piggybacking is used.
- P/F is for dual purpose. If it set to 1, it can mean either poll or final.
- It means poll when the frame is sent by a primary station to secondary.
- It means Final when frame is sent by a secondary to primary.



HDLC: Frames

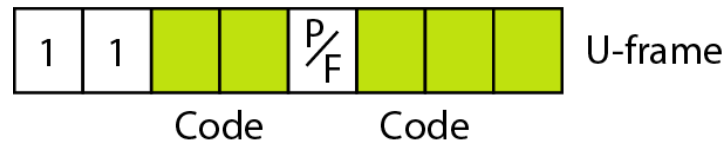
- Control field for **S-frame**
- Used for flow and error control whenever piggybacking is either impossible or inappropriate
- Receive ready (RR), Receive not ready (RNR), Reject (REJ) Selective reject (SREJ)



Code		Command
00	RR	Receive ready
01	REJ	Reject
10	RNR	Receive not ready
11	SREJ	Selective-reject

HDLC: Frames

- Control field for **U-frame**(exchange session management & control information between connected devices).
- Contains information field, but one used for system management, not user data.
- U-frames are divided into two sections: a 2-bit prefix before the P/F bit and a 3-bit suffix after P/F bit.

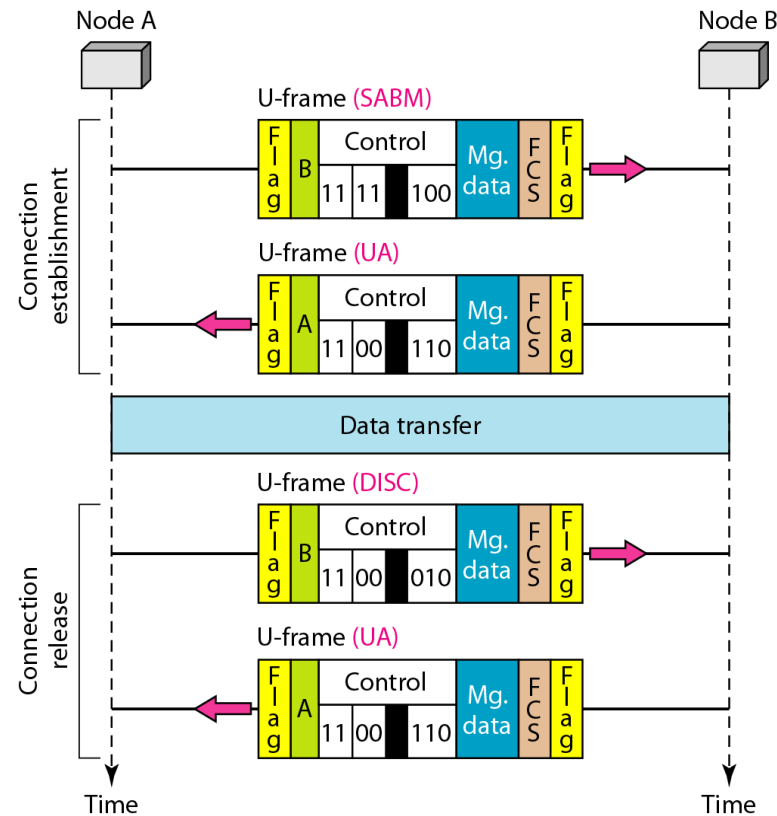


HDLC: Frames

<i>Code</i>	<i>Command</i>	<i>Response</i>	<i>Meaning</i>
00 001	SNRM		Set normal response mode
11 011	SNRME		Set normal response mode, extended
11 100	SABM	DM	Set asynchronous balanced mode or disconnect mode
11 110	SABME		Set asynchronous balanced mode, extended
00 000	UI	UI	Unnumbered information
00 110		UA	Unnumbered acknowledgment
00 010	DISC	RD	Disconnect or request disconnect
10 000	SIM	RIM	Set initialization mode or request information mode
00 100	UP		Unnumbered poll
11 001	RSET		Reset
11 101	XID	XID	Exchange ID
10 001	FRMR	FRMR	Frame reject

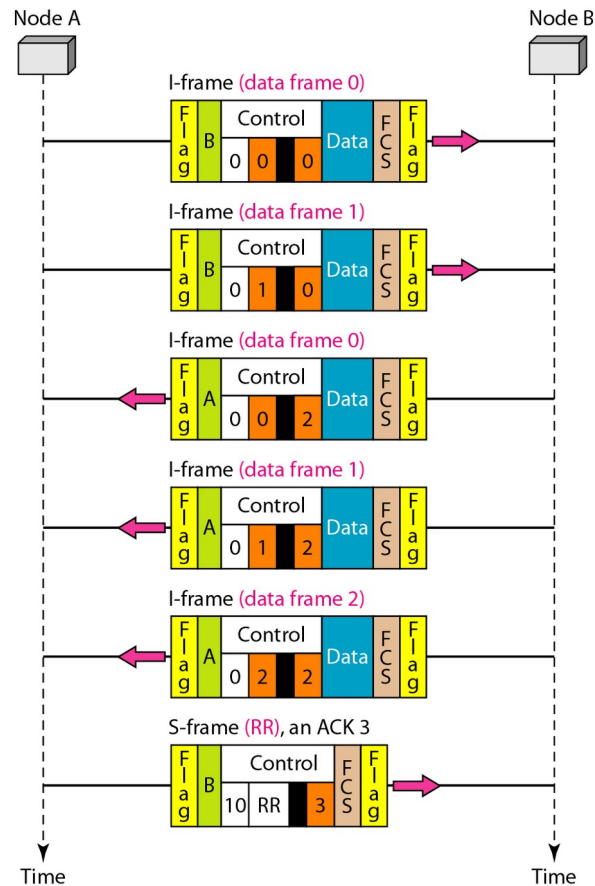
HDLC: Example 1

- Connection and disconnection



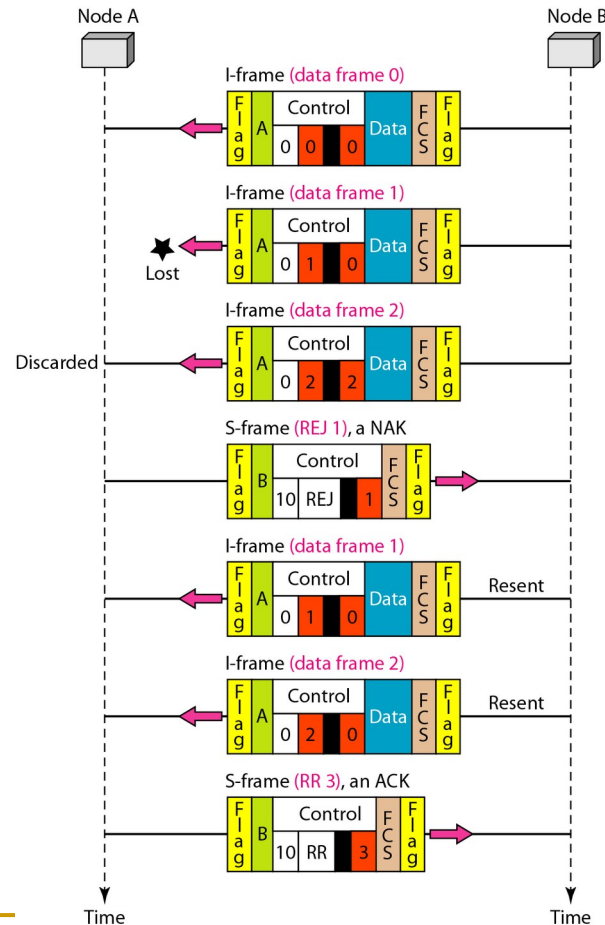
HDLC: Example 2

- Piggybacking without error

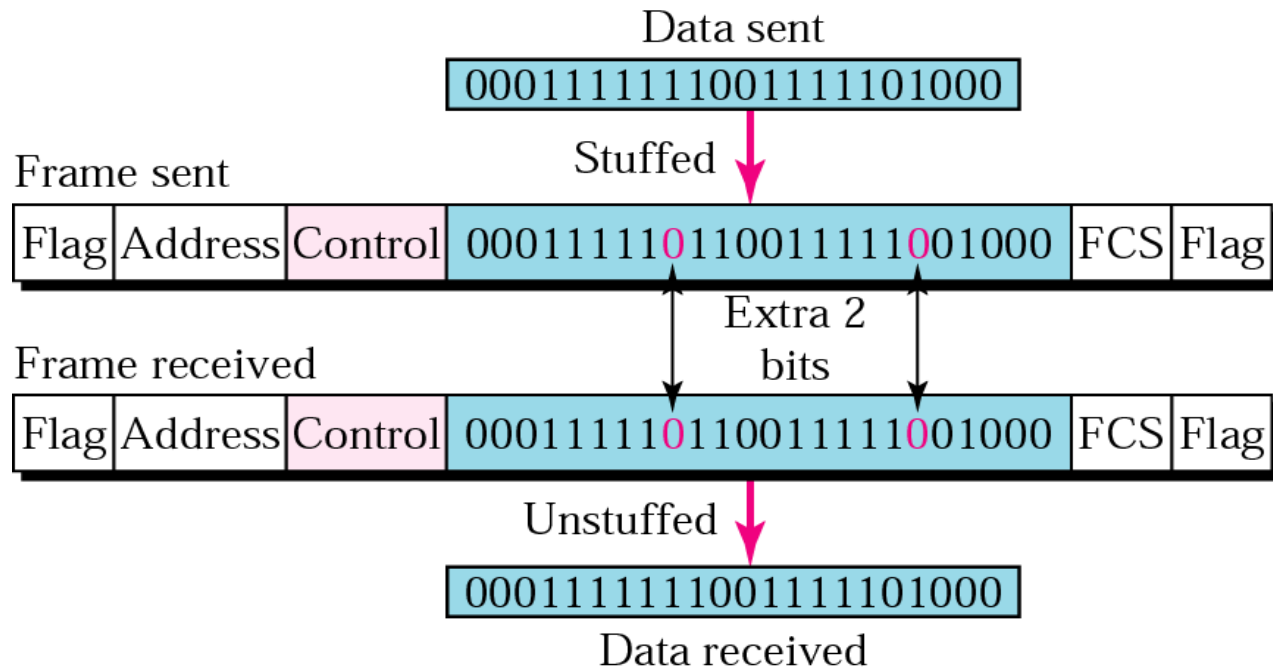


HDLC: Example 3

- Piggybacking with error



HDLC: Bit Stuffing and Unstuffing



Example Data Link Protocols:

***The Data Link Layer in the Internet** - LANs use broadcast protocols (Ethernet) to connect many hosts but interconnection of LANs into larger networks is primary done through routers running a point-to-point protocol. In the case of the Internet, point-to-point protocol is also used to connect host to router in the form of an Internet Service Provider. Two point-to-point protocols widely used on the Internet are SLIP and PPP.*

***SLIP** – **Serial Line Internet Protocol**- Designed to connect hosts to the internet over serial communications.*

***SLIP** – Serial Line Internet Protocol*

- *Designed to connect hosts to the Internet over serial communications.*
- *Send raw IP packets over serial (possibly modem) line with C0 hex at end for framing, if occurs in IP packet uses character stuffing.*
- *Recent versions use TCP and IP header compression by omitting when multiple packets going to same destination.*
- *No error detection/correction, responsibility of higher layers.*
- *Supports only IP.*
- *No authentication (though could be handled by higher layers).*
- *Fixed IPs, both must know each others in advance (not a problem if yours and ISP never change).*

Drawbacks of SLIP

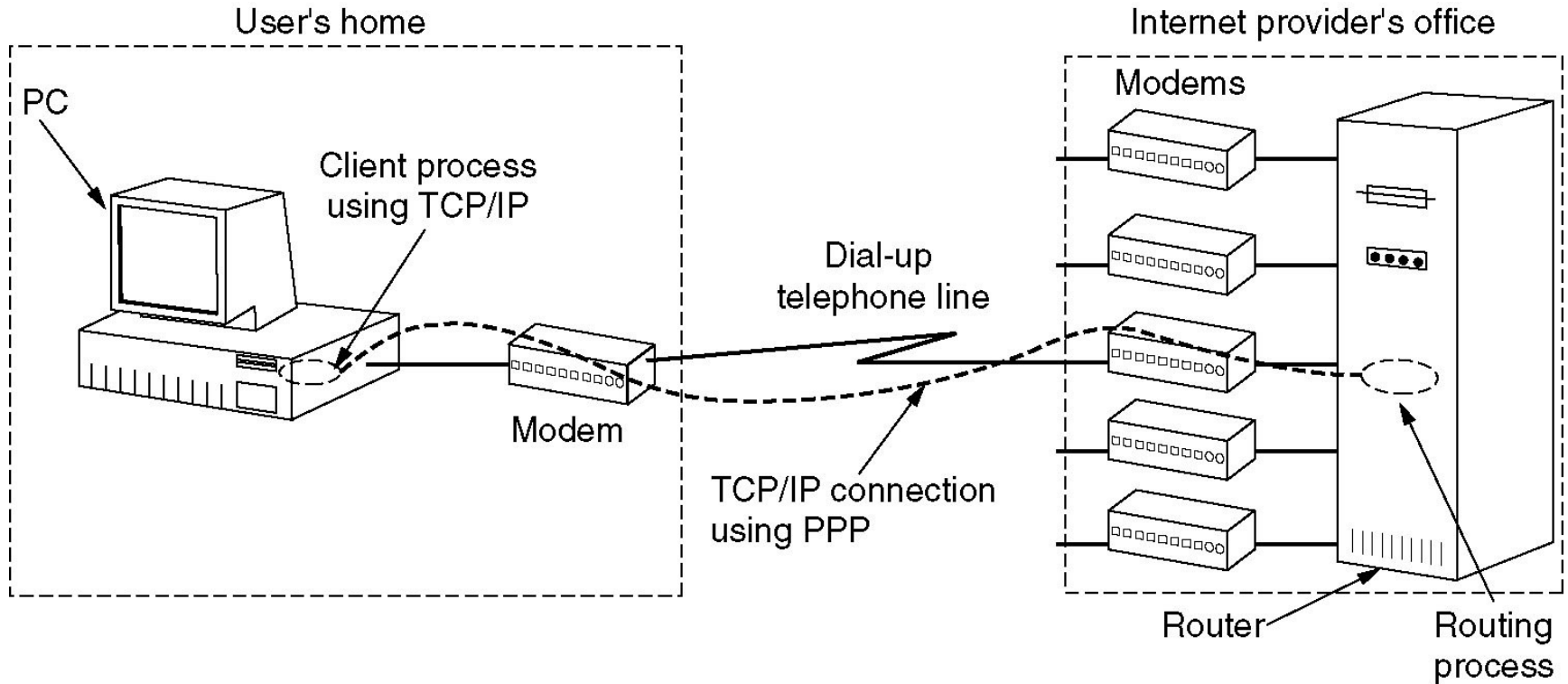
- *SLIP does not do error detection and correction*
 - *SLIP supports only IP*
 - *Each side must know others IP address in advance*
 - *SLIP does not provide any form of authentication*
 - *SLIP is not approved as internet standard*
-

PPP - Point-to-Point Protocol

- *Fixes many of SLIPs problems and is an official Internet protocol*
- *Used in Dial-up connection between residential host and ISP*
- *Framing*
- *Byte stuffing*
- *Connection oriented*
- *Error detection by checksum.*
- *Permits authentication.*
- ***LCP**: It is responsible for establishing the connection and releasing the connection*
- ***NCP**: Network Control Protocol – It provides all the services of Network layer*

-
- *Overcomes all deficiencies of SLIP.*
 - *A better framing method.*
 - *Frame format supports error detection.*
 - *Provide with a Link Control protocol (LCP), for bringing lines up, testing them, negotiating options, and bringing them down back.*
 - *Have different NCP (Network Control Protocol) for each different network protocol.*
-

The Data Link Layer in the Internet



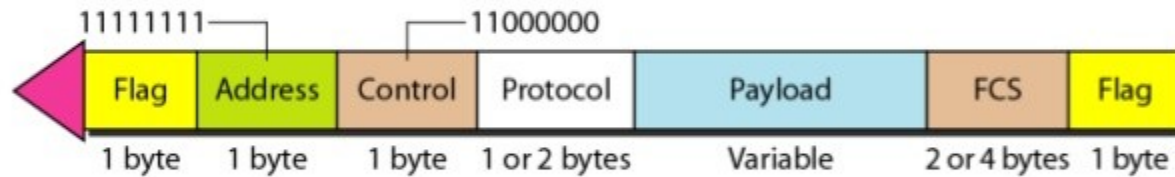
A home personal computer acting as an internet host.

PPP operation

- *PC calls the provider's router via a modem.*
 - *After establishment of the connection PC sends a series of LCP packets.*
 - *Then a series of NCP packets are sent to configure the network layer.*
 - *Now PC is ready to send and receive IP packets.*
 - *After finishing again NCP packets are sent to tear down the network layer connections.*
 - *Finally LCP packets are used to shutdown the connection.*
 - *PC releases the physical connection via the modem.*
-

PPP - Point-to-Point Protocol

Figure 11.32 *PPP frame format*

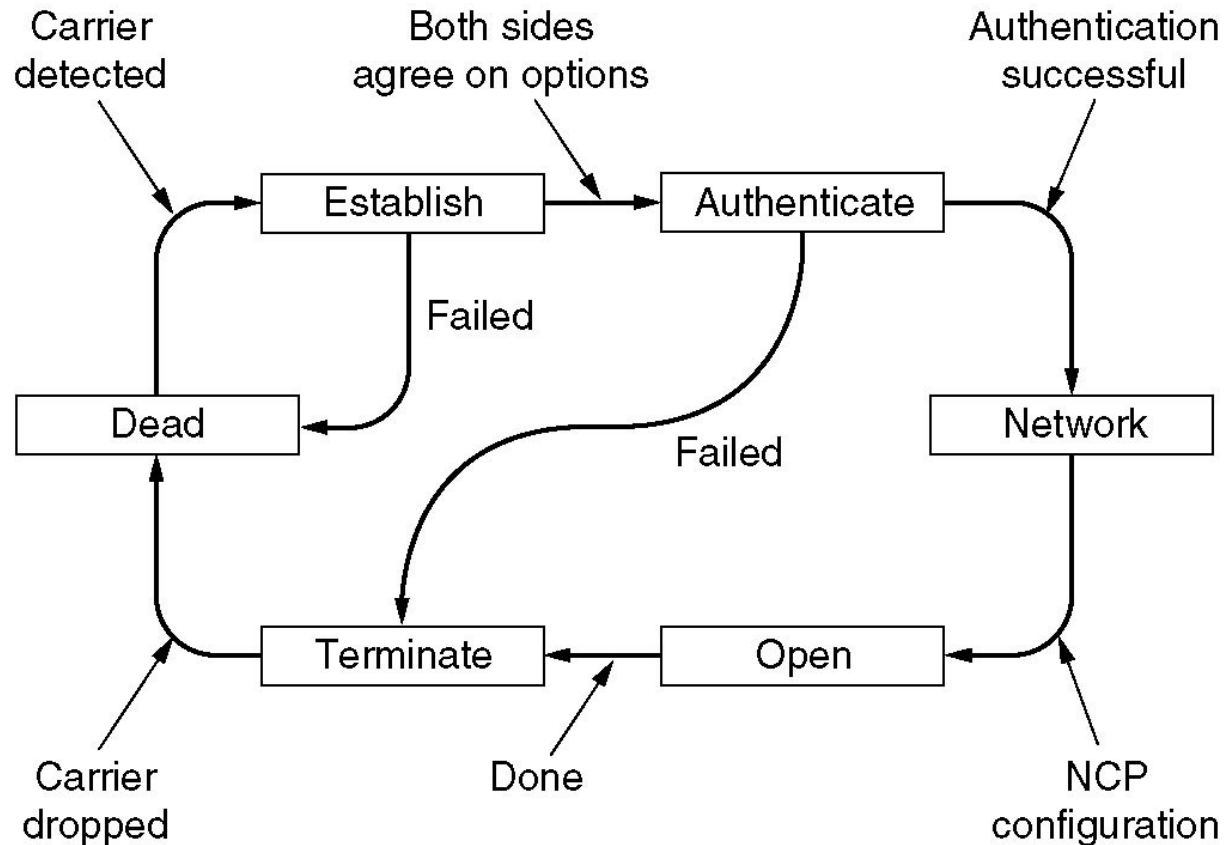


-
- **Flag:** A PPP frame starts and ends with a 1-byte flag with the bit pattern 01111110. It is a byte-oriented protocol. The flag is treated as a byte.
 - **Address:** It is a constant value and is set to 11111111 (broadcast address). During negotiation, the two parties may agree to omit this byte.
 - **Control:** 11000000 (imitating U-frame in HDLC). PPP does not provide any flow control. Error control is also limited to error detection.
 - **Protocol:** It defines what is being carried in the data field: either user data or other information.
 - **Payload field:** It carries either the user data or other information. The default of max..1500 bytes.
 - **FCS:** 2-bytes or 4-bytes standard CRC.
-

Byte Stuffing:

- PPP is a byte oriented protocol.
 - The flag in PPP is a byte and needs to be escaped whenever it appears in the data section of the frame.
 - The esc byte is 0111101, means that every time the flag like pattern appears in the data, this extra byte is stuffed to tell the receiver that the next byte is not a flag.
-

PPP – Point to Point Protocol



A simplified phase diagram for bring a line up and down.

PPP – Point to Point Protocol

Name	Direction	Description
Configure-request	I → R	List of proposed options and values
Configure-ack	I ← R	All options are accepted
Configure-nak	I ← R	Some options are not accepted
Configure-reject	I ← R	Some options are not negotiable
Terminate-request	I → R	Request to shut the line down
Terminate-ack	I ← R	OK, line shut down
Code-reject	I ← R	Unknown request received
Protocol-reject	I ← R	Unknown protocol requested
Echo-request	I → R	Please send this frame back
Echo-reply	I ← R	Here is the frame back
Discard-request	I → R	Just discard this frame (for testing)

- The LCP frame types.